
Towards Functional Correctness of Code Models with Selective Generation

Jaewoo Jeong¹ Taesoo Kim^{2,3} Sangdon Park^{1,4}

Abstract

The hallucination of code generation models hinders their applicability to systems requiring higher safety standards. One critical bottleneck in addressing code hallucination is the difficulty of identifying the functional correctness of generated code, due to its unnatural form. We address this core bottleneck by automatically generating unit tests using dynamic code analysis tools, leveraging the *executable nature* of code. Accordingly, we propose a *selective code generator* that abstains from uncertain generations – based on the functional correctness evaluated by generated unit tests – to theoretically control the correctness among non-abstained answers, *i.e.*, the false discovery rate. Finally, we propose to use generated unit tests in evaluation as well as in learning for precise code evaluation, calling this paradigm *FuzzEval*. We demonstrate the efficacy of our method along with the controllability of code hallucination and reasonable selection efficiency.

1. Introduction

Large language models (LLMs) have recently been proven to be performant in various tasks, including question-answering, summarization, mathematical reasoning, and algorithmic problem-solving (Brown et al., 2020; Li et al., 2022; Touvron et al., 2023; Ahn et al., 2024). Along with language generation, code generation is closely related but has its own benefit as a core task for addressing many applications, including mathematical solving via program of thought, security patch generation, and general program development (Chen et al., 2023b; Hossain et al., 2024; Kim et al., 2025).

Recent developments on large code models have primarily focused on enhancing model performance (Guo et al., 2025;

OpenAI et al., 2025) – thereby *indirectly controlling* the functional hallucination. However, *direct control* methods to address *functional hallucination in code generation*, *i.e.*, a situation where generated code does not satisfy a desired functionality, remain unexplored.

In contrast to hallucination control in code, heuristic and certified methods for hallucination control in natural language generation have been extensively studied beyond enhancing the model performance. For example, the language hallucination is heuristically measured by generating multiple answers and checking the consensus among them (Manakul et al., 2023; Kuhn et al., 2023). As more sophisticated methods, hallucination is carefully controlled by certified methods, including conformal prediction (Vovk et al., 2005) and selective prediction (Geifman & El-Yaniv, 2017), providing certified ways to mitigate language hallucination (Quach et al., 2024; Mohri & Hashimoto, 2024; Lee et al., 2024).

We claim that a critical bottleneck in mitigating code hallucination mainly stems from the intricacy of identifying functional equivalence between two code snippets due to the unnatural form of code. In natural language, textual entailment (Bowman et al., 2015) is the main building block in measuring the semantic correctness of an answer, *i.e.*, an answer is correct if a question-associated context entails the answer. Given this definition on the correctness between two sentences, humans can manually annotate entailment labels to learn entailment-predicting models (Williams et al., 2018). However, this is challenging in code as it is not easy for humans to decide whether one code entails another due to its complex, un-natural structure for obtaining entailment labels. This can be partially mitigated by constructing a few unit tests (Chen et al., 2021; Austin et al., 2021; Casano et al., 2022) and executing it to check discrepancies in output, while limited to a small number of unit tests.

To address these challenges, we build on prior programming language literature to re-define entailment for code generation. We further exploit the *executable property of code* by automatically generating unit tests through code analysis tools. In particular, we leverage fuzzing methods (Miller et al., 1990), one of the practical code analysis tools, to automatically generate unit tests.

Given the code entailment definition and generated unit

¹CSE, POSTECH ²SCS&SCP GaTech ³Microsoft ⁴GSAI, POSTECH. Correspondence to: Sangdon Park <sangdon@postech.ac.kr>.

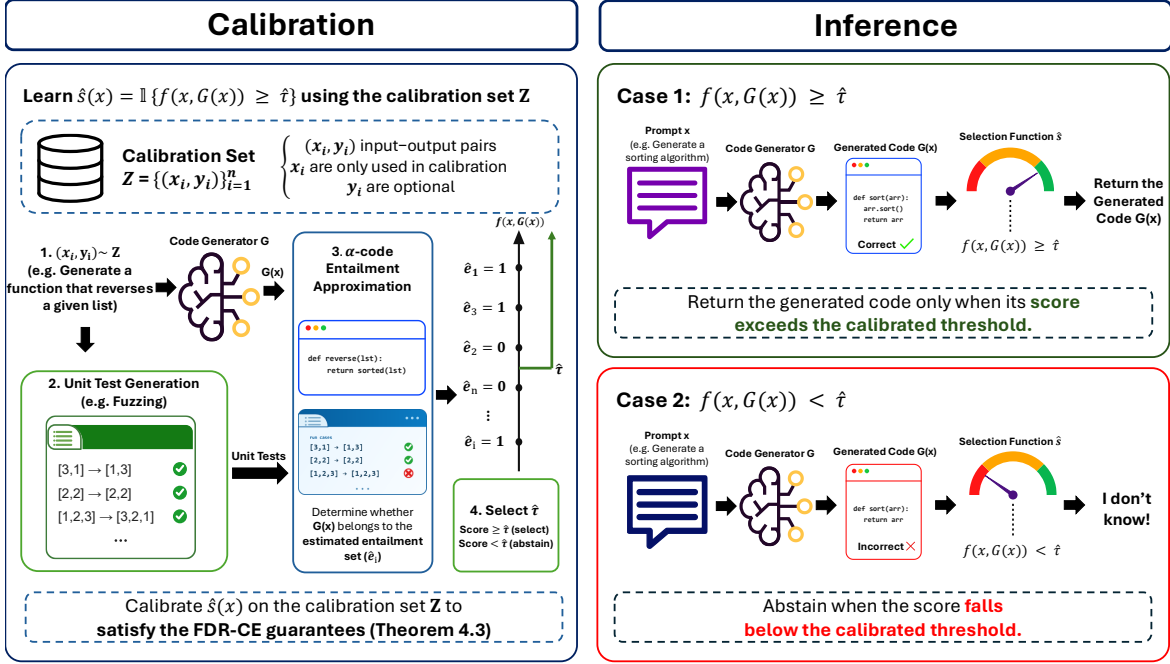


Figure 1. Overview of our proposed selective code generation. We leverage an abstaining option to selectively generate code to control the rate of hallucination in an FDR. Our selective generator learns a selection function by leveraging dynamic code analysis tools to automatically generate unit tests and use them as a calibration set for the selection function and also as a test set for evaluation.

tests, we propose certified selective code generation for code hallucination control. This includes a learning algorithm of a selective generator for code, a post-processor of an original generator, which provides a controllability guarantee on the rate of hallucination in a false discovery rate (FDR), *i.e.*, a learned selective generator provides a desired or minimum level of hallucination. The learning algorithm mainly leverages fuzzing for checking the correctness of code in a self-supervised manner for supervised learning in selective code generators. Finally, we leverage fuzzing for code evaluation as well as selective generator learning. We claim that automatically generated unit tests provide rigorous evaluation, calling this evaluation paradigm *FuzzEval* to distinguish from *HumanEval* (Chen et al., 2021).

The main contribution of our work lies in the selective generation learning algorithm that controls the hallucination rate. We demonstrate the efficacy of our algorithm over open and closed code generators under diverse experiment setups, including four code generators, four datasets, four programming languages, and diverse baselines. Our results demonstrate the hallucination-controllability with reasonable efficiency, and further demonstrate the benefits of automatically generated unit tests. We will release code and dataset at <https://github.com/trustml-lab/selective-code-generation>.

1.1. Related Work

Execution Based Code Correctness Evaluation. Popular code generation benchmarks, such as HumanEval (Chen

et al., 2021), APPS (Hendrycks et al., 2021), or MBPP (Austin et al., 2021) evaluate the functional correctness of a generated code snippet by executing unit tests. However, generating unit tests for such evaluation purposes is a costly task. Automated unit test generation has been explored in recent work. LLMs themselves have been used to improve unit tests (Alshahwan et al., 2024). EvalPlus (Liu et al., 2023) and SemCoder (Ding et al., 2024) adopt a type-aware input mutation strategy based on LLM-generated seeds, while Mercury (Du et al., 2024) leverages LLMs to generate random test case generators for evaluation split. Our work complements these works by generating unit tests via dynamic analysis tools, *e.g.*, fuzzing tools, that have been extensively studied and validated within the computer security community, to explore execution paths.

Previous works that leverage unit tests for evaluation commonly use the pass@k metric to assess functional correctness (Liu et al., 2023; Ding et al., 2024). EvalPlus (Liu et al., 2023) reports a drop in pass@k metric with larger test suites indicating that the metric is sensitive to the number and quality of unit tests. Furthermore, pass@k cannot consider partially correct programs. Our work addresses this by introducing the FDR-CE metric with a statistical guarantee.

Code Generation and Hallucination. Code generation is the task of generating a program that satisfies given functional specifications. The main challenge for this task is the generation of functionally incorrect code, often referred to as *code hallucination* in the previous literature (Tian et al.,

2025; Zhang et al., 2025).

Recent work has mostly focused on improving the functionality itself. DeepSeek-R1 (Guo et al., 2025), OpenAI-IOI (OpenAI et al., 2025) leverage reinforcement learning based finetuning, resulting in highly performant code generation models. CodeT (Chen et al., 2023a), TiCoder (Fakhoury et al., 2024), 3DGen (Fakhoury et al., 2025), MPSC (Huang et al., 2024), S* (Li et al., 2025), and CodeRL (Le et al., 2022) rank generated solutions during the inference or evaluation phase to determine which outputs to adopt. AlphaCode (Li et al., 2022) leverages both finetuning and a ranking mechanism to improve performance. The key distinction of our work lies in directly controlling the rate of falsely generated code (FDR-CE), making the underlying code generator more trustworthy. Furthermore, our work is applicable on top of each method providing an upper bound on the FDR-CE (e.g., DeepSeek result in Figure 2(c) shows that our method can provide an additional statistical guarantee when applied on top of GRPO based fine-tuning (Guo et al., 2025)).

The methods mentioned above leveraged unit tests to improve performance. The ranking-based methods (Chen et al., 2023a; Fakhoury et al., 2024; 2025; Huang et al., 2024; Li et al., 2025; Le et al., 2022) leverage unit tests to rank sampled solutions and select the best code, whereas fine-tuning based methods (Guo et al., 2025; OpenAI et al., 2025) leverage unit tests to provide execution feedback for reinforcement learning. Unlike prior work, our method leverages unit tests to determine α -code entailment and apply a selective generation learning algorithm.

Selective Generation. Selective prediction abstains from answering if a model is not confident of the answer, from which it controls the risk at a desirable level. This method can be applicable to various tasks. Selective classification for deep learning (Geifman & El-Yaniv, 2017) considers classification tasks. Selective text generation (Lee et al., 2024) is applied to a language generation task by introducing textual entailment to control hallucination. We extend selective generation for code by leveraging an executable property of code, while still controlling hallucination.

2. Preliminary

We introduce preliminaries on textual entailment for measuring correctness between two answers, selective generation to control the rate of hallucination, and dynamic code analysis via fuzzing for learning and evaluating functional correctness of code snippets.

Selective Generation. Language models suffer from generating hallucinated facts. Recently, certified ways to control the rate of hallucination in language models have been proposed (Quach et al., 2024; Mohri & Hashimoto, 2024; Lee

et al., 2024). Among them, selective generation, which extends traditional selective classification (Geifman & El-Yaniv, 2017), provides a way to control the rate of hallucination defined in terms of a false discovery rate with entailment (FDR-E) that leverages textual entailment to measure the correctness of two answers. In particular, a selective generator $\hat{S}(\mathbf{x})$ given a question $\mathbf{x}$ returns a generated answer $G(\mathbf{x})$ from a language model or abstains from answering by returning “I don’t know” (IDK). The FDR-E of this selective generator with respect to a true answer $\mathbf{y}$ is defined as $\mathcal{R}(\hat{S}) := \mathbb{P}\{\hat{S}(\mathbf{x}) \notin E(\mathbf{y}) \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\}$. Here, E is an entailment set that contains entailed answers, i.e., $E(\mathbf{y}) := \{\bar{\mathbf{y}} \mid \bar{\mathbf{y}} \text{ entails } \mathbf{y}\}$ (where a reverse relation is also valid), so $\hat{S}(\mathbf{x}) \in E(\mathbf{y})$ means that $\hat{S}(\mathbf{x})$ entails $\mathbf{y}$. In semi-supervised selective generation (Lee et al., 2024), a learning algorithm leverages an estimated entailment set $\hat{E}$ learned from the handful of entailment labels, where $\hat{E}$ is used as a pseudo-labeling function for entailment labels. Based on the estimated entailment set, the following surrogate of the FDR-E is considered: $\hat{\mathcal{R}}(\hat{S}) := \mathbb{P}\{\hat{S}(\mathbf{x}) \notin \hat{E}(\mathbf{y}) \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\}$. The algorithm leverages the relation between $\mathcal{R}(\hat{S})$ and $\hat{\mathcal{R}}(\hat{S})$ to bound $\mathcal{R}(\hat{S})$, as shown in the following lemma.

Lemma 2.1. (Lee et al., 2024) $\mathcal{R}(\hat{S})$ is decomposed in $\mathcal{R}(\hat{S}) := \mathbb{P}_{\mathcal{D}_S}\{e = 0, \hat{e} = 1\} - \mathbb{P}_{\mathcal{D}_S}\{e = 1, \hat{e} = 0\} + \hat{\mathcal{R}}(\hat{S})$, where $\mathbb{P}_S\{\cdot\} := \mathbb{P}\{\cdot \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\}$, $e := \mathbb{1}(\hat{S}(\mathbf{x}) \in E(\mathbf{y}))$, and $\hat{e} := \mathbb{1}(\hat{S}(\mathbf{x}) \in \hat{E}(\mathbf{y}))$.

Importantly, the above lemma assumes that the true label $\mathbf{y}$ is given. But, in code generation it is equivalent to considering a unit test generator $\mathcal{F}(\mathbf{x})$ where generated unit tests play a role of the true label $\mathbf{y}$.

By controlling the upper bound of three decomposed terms in $\mathcal{R}(\hat{S})$ at a desired level, the algorithm learns a selective generator $\hat{S}$. We leverage this framework to learn $\hat{E}$ via generated unit tests via dynamic code analysis tools. See Appendix A for additional detail.

3. Problem

We consider a learning problem in code generation. In particular, we learn a code generator to control code hallucination from the perspective of *functional correctness* where the generator abstains from answering if it is not certain on the correctness of the generated code. Let $\mathcal{W}$ be a set of tokens, $\mathcal{W}^* := \cup_{i=0}^{\infty} \mathcal{W}^i$, $\mathcal{X} := \mathcal{W}^*$ be a set of input prompts (e.g., problem descriptions) for a code generator, $\mathcal{Y} := \mathcal{W}^*$ be a set of code snippets, $\mathcal{D}$ be a distribution that depends on prompt and code pairs $\mathcal{X} \times \mathcal{Y}$ along with other random sources, and $G : \mathcal{X} \rightarrow \mathcal{Y}$ be a code generator.

To control the hallucination rate of a code generator G , we consider the following selective generator $\hat{S} : \mathcal{X} \rightarrow \mathcal{Y} \cup \{\text{IDK}\}$ (Geifman & El-Yaniv, 2017; Lee et al., 2024):

$\hat{S}(\mathbf{x}) := \begin{cases} G(\mathbf{x}) & \text{if } \hat{s}(\mathbf{x}) = 1 \\ \text{IDK} & \text{otherwise} \end{cases}$, where IDK is a short-hand for “I don’t know” and $\hat{s} : \mathcal{X} \rightarrow \{0, 1\}$ is a selection function. Here, we consider a setup that the target code generator G is given and we learn a selection function $\hat{s}$.

We mainly learn the selective generator under the independent and identically distributed (i.i.d.) assumption by controlling a false discovery rate (FDR) for code. In particular, we consider the risk definition of a selective generator $\hat{S}$ via an FDR with a relation R , i.e.,

$$\mathcal{R}_R(\hat{S}) := \mathbb{P} \left\{ (\hat{S}(\mathbf{x}), \mathcal{F}(\mathbf{x})) \notin R \mid \hat{S}(\mathbf{x}) \neq \text{IDK} \right\}, \quad (1)$$

where $\mathcal{F}(\mathbf{x})$ is a unit test generator from a problem description $\mathbf{x}$ and the probability is taken over $\mathbf{x} \sim \mathcal{D}$ and the randomness of $\hat{S}$ and $\mathcal{F}$. We measure the ratio of failure, i.e., the ratio that generated code $G(\mathbf{x})$ does not have a relation R with respect to a unit test generator $\mathcal{F}(\mathbf{x})$, among not-abstaining cases.

Given the learning objective in the FDR, we find a learning algorithm $\mathcal{A}$ for $\hat{S}$ such that given a calibration set $\mathbf{Z}$ with $|\mathbf{Z}| = n$, the learned selective generator $\hat{S} := \mathcal{A}(\mathbf{Z})$ controls a desired risk level ε with probability at least $1 - \delta$, i.e., $\mathbb{P} \{ \mathcal{R}_R(\mathcal{A}(\mathbf{Z})) \leq \varepsilon \} \geq 1 - \delta$, where the probability is taken over $\mathbf{Z} \sim \mathcal{D}^n$. Here, the main challenges include (1) designing a correctness relation R for code generation and (2) finding a learning algorithm with the above PAC-style controllability guarantee, while maximizing *selection efficiency*, i.e., $\mathbb{P} \{ \hat{S}(\mathbf{x}) \neq \text{IDK} \}$, which are addressed in the following section.

4. Method: Selective Code Generation

We introduce a definition of code entailment in Section 4.1, followed by a risk definition in Section 4.2. Section 4.3 and 4.4 present the theoretical bound. Our FDR-controlling algorithm is described in Section 4.5, followed by its controllability guarantee in Section 4.6. Lastly, we highlight the importance of evaluation via fuzzing in Section 4.7.

4.1. Code Entailment

Measuring the functional correctness of a generated code snippet is a challenging task. In particular, the functionality of generated code is evaluated using only a limited set of manually chosen unit tests (Chen et al., 2021). We re-define the concept of *code entailment* that leverages automated unit test generators (e.g., LLMs or dynamic code analysis tools) along with manual unit tests to define the functional correctness with larger unit tests.

To this end, let $\mathcal{U}$ and $\mathcal{V}$ be sets of input and output states for all code snippets, respectively. A unit test generator $\mathcal{F} : \mathcal{X} \rightarrow \Delta(\mathcal{U} \times \mathcal{V})$ returns a pair of input and output states

from a problem description $\mathbf{x}$, i.e., $(\mathbf{u}, \mathbf{v}) \sim \mathcal{F}(\mathbf{x})$. We then introduce the definition of α -code entailment by leveraging the unit test generator $\mathcal{F}$.

Definition 4.1 (α -code entailment). A unit test generator $\mathcal{F}$ on $\mathbf{x}$, $\mathcal{F}(\mathbf{x})$, α -entails $\hat{\mathbf{y}} \in \mathcal{Y}$ if an *expected functional correctness* of $\hat{\mathbf{y}}$ with respect to $\mathcal{F}$, i.e., $\mathbb{P} \{ \hat{\mathbf{y}}(\mathbf{u}) = \mathbf{v} \}$, satisfies

$$\mathbb{P} \{ \hat{\mathbf{y}}(\mathbf{u}) = \mathbf{v} \} \geq 1 - \alpha, \quad (2)$$

where the probability is taken over $(\mathbf{u}, \mathbf{v}) \sim \mathcal{F}(\mathbf{x})$.

We formally re-define the concept of *probabilistic tests* originally introduced by Massalin (1987). Our definition differs from prior work, Claessen & Hughes (2000); McKeeman (1998); Chen & Su (2015), which focuses on identifying counterexamples for program inequivalence. Also, our definition differs slightly from program equivalence definitions in Jakobs & Wiesner (2022); Lahiri et al. (2012); Felsing et al. (2014), as we use α to accommodate different code generator quality and enhance practicality.

In code generation, the α -code entailment provides a foundation for measuring functional correctness. In particular, suppose that we have a target code snippet $\mathbf{y}$ for intuitive explanation. Then, to measure the functional correctness between two code snippets $\mathbf{y}$ and $\hat{\mathbf{y}}$, we need to check whether two code snippets $\mathbf{y}$ and $\hat{\mathbf{y}}$ have the same output state for all input states. To check this bidirectional equivalence relation, we first consider the one-directional definition via code entailment by checking whether code $\hat{\mathbf{y}}$ satisfies all input and output pairs for code $\mathbf{y}$, equivalently unit tests from $\mathcal{F}(\mathbf{x})$ where $\mathbf{y}$ is reference code for a problem $\mathbf{x}$, following Definition 4.1. By considering entailment from $\mathbf{y}$ to $\hat{\mathbf{y}}$ and vice versa, we can eventually define the functional equivalence. In this paper, instead of analyzing the bidirectional equivalence, we only consider the one-directional relaxed notion of correctness via code entailment, which suffices for code generation, e.g., we can say that $\hat{\mathbf{y}}$ is correct if it contains all functionalities of $\mathbf{y}$ along with other functionalities. The example of α -entailment is presented in Table 5.

4.2. False Discovery Rate via Code Entailment

We define a relation for the FDR risk in (1) via α -code entailment. We first denote the set of α -entailment code snippets of $\mathbf{x}$ by $E_\alpha(\mathbf{x})$, i.e., $E_\alpha(\mathbf{x}) := \{ \hat{\mathbf{y}} \mid \mathbb{P} \{ \hat{\mathbf{y}}(\mathbf{u}) = \mathbf{v} \} \geq 1 - \alpha \}$, which approximately corresponds to the set of all code snippets that have the most functionalities described by unit tests from $\mathcal{F}(\mathbf{x})$. By the definition of the α -code entailment, $\hat{\mathbf{y}} \in E_\alpha(\mathbf{x})$ implies that $\mathcal{F}(\mathbf{x})$ α -entails $\hat{\mathbf{y}}$.

Using the same definition of $E_\alpha(\mathbf{x})$, we further define the correctness relation between two code snippets as follows: $R_\alpha := \{ (\hat{\mathbf{y}}, \mathcal{F}(\mathbf{x})) \mid \hat{\mathbf{y}} \in E_\alpha(\mathbf{x}) \}$. Then, from (1), we define the FDR with code entailment relation R_α (FDR-

CE). Equivalently, we use the following FDR-CE definition: $\mathcal{R}_\alpha(\hat{S}) := \mathbb{P}\{\hat{S}(\mathbf{x}) \notin E_\alpha(\mathbf{x}) \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\}$.

4.3. Code Entailment Estimation

In natural languages, an entailment relation between two sentences can be easily obtained by human annotators. However, identifying functionalities between two code snippets by humans (*i.e.*, deciding whether $\hat{S}(\mathbf{x}) \notin E_\alpha(\mathbf{x})$ or not) is challenging due to the un-natural form of programming languages. We overcome this challenge with unit tests that exploit an *executable property of code*.

Inspired by (Lee et al., 2024), we estimate E_α and use it as a pseudo-labeling function as the exact E_α is difficult to obtain. In particular, recall the expected functional correctness $\mathbb{P}\{\bar{\mathbf{y}}(\mathbf{u}) = \mathbf{v}\}$ between unit tests from a test generator on $\mathbf{x}$, $\mathcal{F}(\mathbf{x})$, and a generated code snippet $\bar{\mathbf{y}}$ from $\mathbf{x}$. Considering that the input-output pairs $(\mathbf{u}, \mathbf{v})$ are independently drawn from the test generator $\mathcal{F}(\mathbf{x})$, we estimate the lower bound of the expected functional correctness by using the standard binomial tail bound. Specifically, let the lower binomial tail bound $\hat{L}$ of $\mathbb{P}\{\bar{\mathbf{y}}(\mathbf{u}) = \mathbf{v}\}$ be $\hat{L}(\mathcal{F}(\mathbf{x}), \bar{\mathbf{y}}, n_{\mathbf{x}}, \varepsilon_E) := \hat{L}_{\text{Binom}}(\hat{k}; n_{\mathbf{x}}, \varepsilon_E)$, where $n_{\mathbf{x}}$ is the number of samples, $\mathbf{U}_{\mathbf{x}} \sim \mathcal{F}(\mathbf{x})^{n_{\mathbf{x}}}$ is a set of generated unit tests, and $\hat{k} := \sum_{(\mathbf{u}, \mathbf{v}) \in \mathbf{U}_{\mathbf{x}}} \mathbb{1}(\bar{\mathbf{y}}(\mathbf{u}) = \mathbf{v})$. Here, $\hat{L}_{\text{Binom}}$ is the lower standard binomial tail bound, where $F(k; n, \theta)$ is a cumulative distribution function of a binomial distribution with n trials and success probability θ , and $\hat{L}_{\text{Binom}}(k; n, \delta) := \sup \{\theta \in [0, 1] \mid 1 - F(k; n, \theta) \leq \delta\} \cup \{0\}$. Then, due to its definition, the lower bound holds with high probability (Clopper & Pearson, 1934) as follows: $\mathbb{P}\{\hat{L}(\mathcal{F}(\mathbf{x}), \bar{\mathbf{y}}, n_{\mathbf{x}}, \varepsilon_E) \leq \mathbb{P}_{\mathbf{y}}\{\bar{\mathbf{y}}(\mathbf{u}) = \mathbf{v}\}\} \geq 1 - \varepsilon_E$, where the probability is taken over $\mathbf{U}_{\mathbf{x}} \sim \mathcal{F}(\mathbf{x})^{n_{\mathbf{x}}}$.

Importantly, we carefully choose reasonably small unit test size $n_{\mathbf{x}}$ for a given $\mathbf{x}$ via Algorithm 1. In particular, sample size for the binomial tail bound is usually given, but we can generate as many samples as we wish by executing a unit test generator $\mathcal{F}$. Here, the number of samples should depend on the difficulty in evaluating the correctness of generated code $\hat{\mathbf{y}}$, *i.e.*, as the generated code is ambiguous to check the α -code entailment, we need more samples to be certain. To this end, we increase the unit test size $n_{\mathbf{x}}$ until the lower bound $\hat{L}$ of an expected functional correctness is larger than $1 - \alpha$ (Line 3) to achieve the expected correctness as well. If the sample size exceeds maximum size n_{max} , the algorithm returns n_{max} (Line 6).

From this lower bound $\hat{L}$, we define an *estimated entailment set* as follows:

$$\hat{E}_{\alpha, \varepsilon_E}(\mathbf{x}) := \left\{ \bar{\mathbf{y}} \mid \hat{L}(\mathcal{F}(\mathbf{x}), \bar{\mathbf{y}}, n_{\mathbf{x}}, \varepsilon_E) \geq 1 - \alpha \right\}. \quad (3)$$

Intuitively, code $\bar{\mathbf{y}} \in \hat{E}_{\alpha, \varepsilon_E}(\mathbf{x})$ likely satisfies $\bar{\mathbf{y}} \in E_\alpha(\mathbf{x})$, meaning $\mathcal{F}(\mathbf{x})$ α -entails $\bar{\mathbf{y}}$ with high probability. Thus, the

FDR-CE based on the estimated entailment set is defined as $\mathcal{R}_{\alpha, \varepsilon_E}(\hat{S}) := \mathbb{P}\{\hat{S}(\mathbf{x}) \notin \hat{E}_{\alpha, \varepsilon_E}(\mathbf{x}) \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\}$. Here, the probability is taken over $(\mathbf{x}, n_{\mathbf{x}}) \sim \mathcal{D}_{\mathcal{X} \times \mathbb{N}}$ and $\mathbf{U}_{\mathbf{x}} \sim \mathcal{F}(\mathbf{x})$, where we simply denote the distribution associated to $(\mathbf{x}, n_{\mathbf{x}}, \mathbf{U}_{\mathbf{x}})$ by $\mathcal{D}$.

It is a good alternative for the original FDR-CE $\mathcal{R}_\alpha(\hat{S})$. In the following, we connect $\mathcal{R}_\alpha$ and $\mathcal{R}_{\alpha, \varepsilon_E}$ in learning.

4.4. False Discovery Rate Bound

We consider the upper bound of the FDR-CE $\mathcal{R}_\alpha(\hat{S})$, which leverages the estimated entailment set. Specifically, from Lemma 2.1, we have $\mathcal{R}_\alpha(\hat{S}) \leq \mathbb{P}_{\mathcal{D}_{\hat{S}}}\{e = 0, \hat{e} = 1\} + \mathcal{R}_{\alpha, \varepsilon_E}(\hat{S})$. Here, $e := G(\mathbf{x}) \in E_\alpha(\mathbf{x})$, $\hat{e} := G(\mathbf{x}) \in \hat{E}_{\alpha, \varepsilon_E}(\mathbf{x})$, and $\mathbb{P}_{\mathcal{D}_{\hat{S}}}\{\cdot\} = \mathbb{P}\{\cdot \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\}$, where the probability is taken over $(\mathbf{x}, n_{\mathbf{x}}, \mathbf{U}_{\mathbf{x}}) \sim \mathcal{D}$. This intuitively suggests that the FDR-CE over the exact entailment set can be approximated by the FDR-CE over the estimated entailment set along with its false entailment rate (FER). Moreover, the FER is related to the correctness of the binomial tail bound, controlled by ε_E . This implies the following key lemma. See Appendix G for a proof.

Lemma 4.2. *For any $\alpha, \varepsilon_E \in (0, 1)$, and $\hat{S}$, we have $\mathcal{R}_\alpha(\hat{S}) \leq \varepsilon_E + \mathcal{R}_{\alpha, \varepsilon_E}(\hat{S})$.*

Next, we use this bound to provide an algorithm for $\hat{S}$, controlling this upper bound at a desired level.

4.5. FDR-CE Control Algorithm

We propose a selective generator learning algorithm for code that controls the FDR-CE. In particular, we consider a scalar-parameterization of the selection function $\hat{s}$, *i.e.*, $\hat{s}(\mathbf{x}) := \mathbb{1}(f(\mathbf{x}, G(\mathbf{x})) \geq \tau)$, where $\tau \in \mathbb{R}$. While the scoring function can be any function that quantifies the confidence on generated code, we employ the standard length-normalized log-probability for generated tokens as the default choice, *i.e.*, $f_{\text{norm}}(\mathbf{x}, G(\mathbf{x})) = \sum_i \ln p_i / |G(\mathbf{x})|$, where p_i is the probability assigned by G to generating the i -th token. Additionally, we exploit hold-out U -unit tests as part of our new scoring function by averaging the f_{norm} with a pass@1 score, *i.e.*, $f_{\text{mixed}}(\mathbf{x}, G(\mathbf{x})) := 0.5 f_{\text{norm}}(\mathbf{x}, G(\mathbf{x})) + 0.5 \frac{\sum_{(u_i, v_i) \sim \mathcal{F}(\mathbf{x})} \mathbb{1}(G(\mathbf{x})(u_i) = v_i)}{U}$. Then, our algorithm searches $\hat{S}$ that closely controls the upper bound in Lemma 4.2 within ε_S by solving the following optimization problem:

$$\min_{\tau \in \mathbb{R}} \tau \text{ subj. to } \varepsilon_E + \hat{U}_{\text{Binom}}\left(\hat{k}; |\hat{\mathbf{Z}}|, \frac{\delta_S}{|\log_2 |\hat{\mathbf{Z}}||}\right) \leq \varepsilon_S, \quad (4)$$

where $\mathbf{Z} \sim \mathcal{D}^n$, $\hat{\mathbf{Z}} := \{(\mathbf{x}, \mathbf{y}, \cdot) \in \mathbf{Z} \mid f(\mathbf{x}, G(\mathbf{x})) \geq \tau\}$, $\hat{k} := \sum_{(\mathbf{x}, \mathbf{y}, \cdot) \in \hat{\mathbf{Z}}} \mathbb{1}(G(\mathbf{x}) \notin \hat{E}_{\alpha, \varepsilon_E}(\mathbf{x}))$, and $\hat{U}_{\text{Binom}}$ is the upper binomial tail bound, similarly defined as the lower bound $\hat{L}_{\text{Binom}}$. Here, the algorithm also returns $\hat{U} := \varepsilon_E + \hat{U}_{\text{Binom}}(\cdot)$, the upper bound of the FDR-CE for the optimal

$\hat{\tau}$. We denote the algorithm solving (4) by $\mathcal{A}_{\text{SCG}}$ and see Algorithm 2 for its implementation details. Appendix M.1 includes guidelines on user-specified parameter selection.

At a high level, the algorithm essentially finds a selective code generator (parametrized by τ) that minimizes τ , to maximize the selection efficiency of the selective generator, under the constraint of controlling the FDR-CE by a desired level ε_S . If the minimization is not feasible, the algorithm returns a selective generator that controls the minimum FDR-CE, indicated by $\hat{U}$.

Automated Unit Test Generation. We consider any unit test generator $\mathcal{F} : \mathcal{X} \rightarrow \mathcal{U} \times \mathcal{V}$ to generate a unit test $(\mathbf{u}, \mathbf{v})$, including LLM-based unit test generators (Liu et al., 2023; Chen et al., 2023a), and manual unit test generation. Here, we adopt a fuzzing method that has been popularized due to its simplicity and efficacy in finding bugs. This method is usually used for exploiting a certain execution to trigger bugs but we rather use it for exploring wider execution paths. In particular, we randomly sample a binary stream s from a seed distribution $\mathcal{D}_U$ and use it as a seed, *i.e.*, $s \sim \mathcal{D}_U$, to arbitrarily assign an input state for code, *e.g.*, randomly initializing input parameters for a function call. The initial state typically fails to lead to an interesting execution path; therefore, the fuzzing method mutates the input seed to explore wider execution paths; *e.g.*, Atheris (Google, 2020) mutates an input to increase code coverage. Moreover, we consider problems $\mathbf{x}$ with reference code $\mathbf{y}$. During this repeated execution of the reference code, multiple input and output state pairs are generated, and we randomly select one of them as our final pair $(\mathbf{u}, \mathbf{v})$ for each seed s . Note that for LLM-based unit test generation or manual unit test generation, we often don’t need reference code $\mathbf{y}$.

4.6. Controllability Guarantee

Our algorithm $\mathcal{A}_{\text{SCG}}$ controls an FDR-CE of a learned selective generator. This is a direct consequence of selective prediction (Geifman & El-Yaniv, 2017; Lee et al., 2024). See a proof in Appendix H.

Theorem 4.3. *For any $\varepsilon_S \in (0, 1)$, $\delta_S \in (0, 1)$, $\alpha \in (0, 1)$, f , $\mathcal{F}$, and $\mathcal{D}$, we have $\mathbb{P}\{\mathcal{R}_\alpha(\hat{S}) \leq \hat{U}\} \geq 1 - \delta_S$, where the probability is taken over $\mathbf{Z} \sim \mathcal{D}^n$ and $(\hat{S}, \hat{U}) := \mathcal{A}_{\text{SCG}}(\mathbf{Z})$.*

This means that $\mathcal{A}_{\text{SCG}}$ finds a selective code generator satisfying a desired FDR-CE ε_S (if $\varepsilon_S \geq \hat{U}$) or minimum level $\hat{U}$ (if $\hat{U} > \varepsilon_S$), without human-feedback on code entailment labels.

4.7. FuzzEval: Evaluation via Fuzzing

We suggest to use automatically generated unit tests via fuzzing in evaluation benchmarks where the reference code snippets are available and fuzzing harnesses are provided. Traditionally, identifying the functional equivalence between desired code and generated code in code generation

has relied on manually obtained unit tests, *e.g.*, HumanEval (Chen et al., 2021). But, as desired code becomes more complex so the number of its execution paths exponentially increases, manually obtaining a sufficient number of unit tests is challenging. To address this bottleneck, we propose to use fuzzing, a dynamic code analysis tool, to automatically generate unit tests for a given code snippet, calling this evaluation paradigm *FuzzEval*.

5. Experiments

We demonstrate the efficacy of our selective code generation on open and closed LLMs in an algorithmic solving task. See Appendix L for additional experiments.

5.1. Setup

Dataset. We considered datasets with coding problems, where each problem has a correct solution. We chose APPS (Hendrycks et al., 2021), Mercury (Du et al., 2024), HumanEval (Chen et al., 2021), and MBPP (Austin et al., 2021) for our task. Each dataset consists of Python programming questions, canonical code solutions, and a few unit tests. Each Python programming question is provided to an LLM as a prompt to generate code, and then the generated code is measured for its functionality by running the unit tests.

We construct APPS-f, Mercury-f, HumanEval-f, and MBPP-f, where for each dataset, we replace the built in unit tests from each problem with automatically generated unit tests via fuzzing for both learning and evaluation. In particular, each question consists of constraints to inputs. We manually post-processed Python programming questions and their solutions of each dataset such that the solution code can be easily called by a fuzzing tool while satisfying the input constraints of questions. Among the post-processed code, we conducted fuzzing and extracted at least 600 input-output pairs as our unit tests for calibration and its evaluation. Additional details can be found in Appendix F.

LLMs. We used three closed LLMs, *i.e.*, GPT-4o (OpenAI, 2024), Gemini 1.5 Pro (Team et al., 2024), GPT-4.1 (OpenAI, 2025) and two open LLMs, *i.e.*, CodeLlama 13B-instruct (Rozière et al., 2024) and DeepSeek-R1 (Guo et al., 2025). Here, we use the following default parameters unless specified: $\varepsilon_S = 0.3$, $\delta_S = 0.1$, $\alpha = 0.35$, $\varepsilon_E = 0.05$, and $n_{\text{max}} = 150$.

Method. We consider seven baseline methods **SCG-EM** (Geifman & El-Yaniv, 2017), **SCG-manual**, **SCG-small**, **SCG-H**, **CodeT** (Chen et al., 2023a), **LDB** (Zhong et al., 2024), and **SFS** (Light et al., 2025) to compare with our method **SCG**.

- **SCG-EM** (Geifman & El-Yaniv, 2017): This baseline is a conventional selective predictor method that compares the generated code and solution code, without measuring its functional correctness.

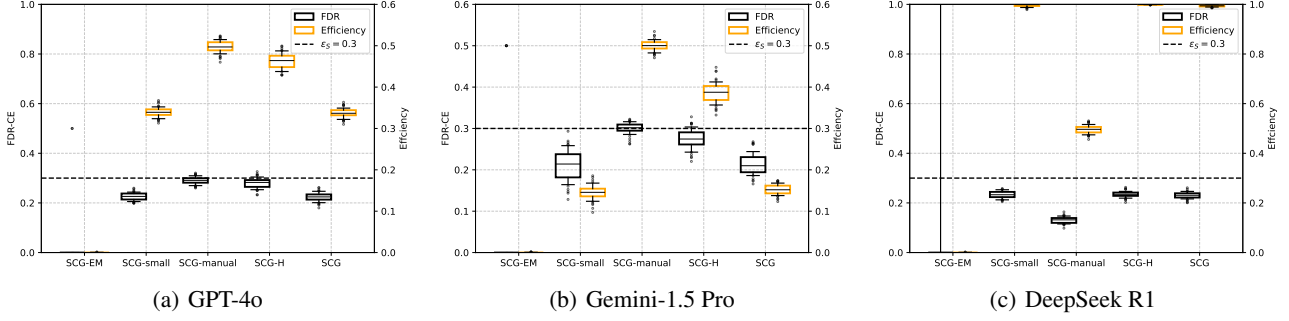


Figure 2. The box plots of the FDR-CE and selection efficiency for various LLMs ($\delta_S = 0.1$, $\varepsilon_S = 0.3$, $\varepsilon_E = 0.05$, $\alpha = 0.35$). See Figure 4(a) for results with CodeLlama 13B Instruct.

Table 1. Comparison results of SCG against baseline methods on different datasets with GPT-4o ($\alpha = 0.35$, $\delta_S = 0.1$, $\varepsilon_E = 0.05$ for all datasets). The FDR-CE satisfying the desired guarantees and the highest efficiency among methods that comply with the FDR-CE guarantees are marked in **bold**.

Methods	w/o Selective Generation	w/ Selective Generation				
	$\tau = -\infty$	SCG-EM	SCG-MANUAL	SCG-SMALL	SCG-H	SCG
APPS-F ($\varepsilon_S = 0.3$)						
1 - PASS@1($\downarrow$)	0.436 $\pm$ 0.012	0.020 $\pm$ 0.098	0.293 $\pm$ 0.014	0.226 $\pm$ 0.015	0.282 $\pm$ 0.020	0.227 $\pm$ 0.017
FDR-CE($\downarrow$)	0.431 $\pm$ 0.011	0.020$\pm$0.099	0.291 $\pm$ 0.014	0.226$\pm$0.015	0.280 $\pm$ 0.020	0.224$\pm$0.018
EFFICIENCY($\uparrow$)	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.497 $\pm$ 0.014	0.339$\pm$0.012	0.463 $\pm$ 0.019	0.337 $\pm$ 0.011
MBPP-F ($\varepsilon_S = 0.4$)						
1 - PASS@1	0.299 $\pm$ 0.022	0.000 $\pm$ 0.000	0.258 $\pm$ 0.040	-	0.301 $\pm$ 0.028	0.304 $\pm$ 0.032
FDR-CE	0.294 $\pm$ 0.027	0.000$\pm$0.000	0.254$\pm$0.041	-	0.296$\pm$0.029	0.300$\pm$0.032
EFFICIENCY	1.000 $\pm$ 0.000	0.001 $\pm$ 0.003	0.499 $\pm$ 0.038	-	0.997$\pm$0.004	0.996 $\pm$ 0.006
HUMAN-EVAL-F ($\varepsilon_S = 0.3$)						
1 - PASS@1	0.185 $\pm$ 0.058	0.000 $\pm$ 0.000	0.142 $\pm$ 0.080	-	0.156 $\pm$ 0.165	0.069 $\pm$ 0.173
FDR-CE	0.207 $\pm$ 0.064	0.000$\pm$0.000	0.156$\pm$0.086	-	0.145 $\pm$ 0.123	0.049$\pm$0.111
EFFICIENCY	1.000 $\pm$ 0.000	0.008 $\pm$ 0.017	0.492$\pm$0.080	-	0.578 $\pm$ 0.401	0.164 $\pm$ 0.118
MERCURY-F ($\varepsilon_S = 0.3$)						
1 - PASS@1	0.174 $\pm$ 0.018	0.000 $\pm$ 0.000	0.138 $\pm$ 0.024	-	0.169 $\pm$ 0.017	0.170 $\pm$ 0.020
FDR-CE	0.170 $\pm$ 0.019	0.000$\pm$0.000	0.133$\pm$0.022	-	0.164$\pm$0.019	0.165$\pm$0.020
EFFICIENCY	1.000 $\pm$ 0.000	0.001 $\pm$ 0.002	0.504 $\pm$ 0.033	-	0.998 $\pm$ 0.003	0.998$\pm$0.002

- **SCG-manual**: This baseline is a simple selective generator that selects the upper $k\%$ of scores for a threshold τ to highlight the importance of controlling the FDR-CE.
- **SCG-small**: This is our method but only using unit tests, provided by the APPS dataset to show the efficacy of generated unit tests via fuzzing. We sampled 21 test cases for each problem to apply our algorithm. See Appendix D for additional details.
- **SCG-H**: This baseline is a heuristic of our method omitting false entailment rate (FER) as in Lemma 4.2. It searches for a selective generator as in (4) with $\varepsilon_E = 0$ (*i.e.*, an ablation method of ours).

Scoring Function. To analyze the effect of calibration on our method **SCG**, we consider four different scoring functions. The detailed explanations on the scoring functions are provided in Appendix E but by default we use the length-normalized scoring function f_{norm} unless specified.

Evaluation. We evaluate our method along with baselines

based on the empirical counterpart of the FDR-CE and selection efficiency from a test set $\mathbf{Z}_t \sim \mathcal{D}^{n_{\text{test}}}$ in (8) and (9), respectively. Interestingly, FDR-CE and 1-PASS@1 (Chen et al., 2021) on selected samples by **SCG** may be asymptotically equivalent when $\alpha \rightarrow 0$ and $n_x \rightarrow \infty$ for PASS@1 (See Appendix J for a discussion).

5.2. Results

We demonstrate the efficacy of our method **SCG** on different models, datasets, and programming languages. In addition, we highlight the benefits of fuzzing, and analyze the effect of calibration. In short, the proposed method satisfies the desired FDR-CE constraint with reasonable efficiency (compared to the maximum efficiency of the underlying generators) where the baseline fails to do so (*i.e.*, **SCG-EM** and **SCG-small** exhibit poor or trivial efficiency in Figure 2(a) and Table 1, respectively; and **SCG-manual** and **SCG-H** violate the FDR-CE constraint in Figure 2(b)).

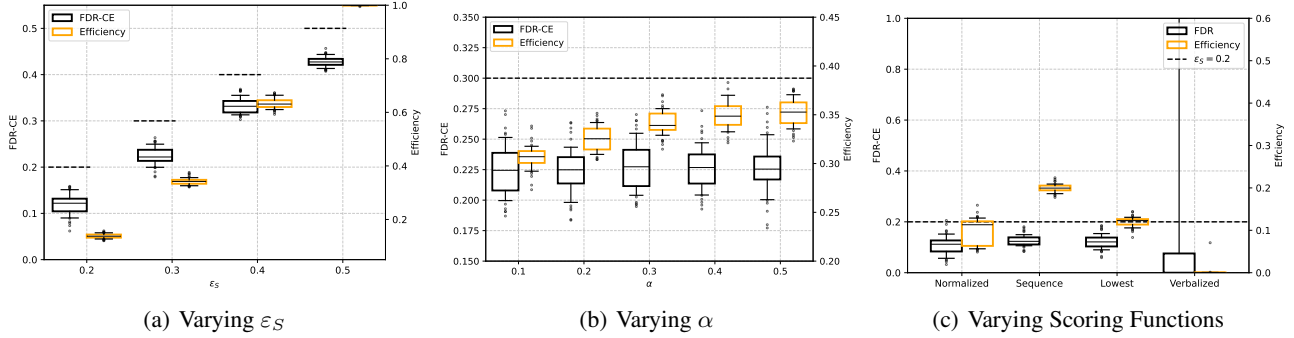


Figure 3. The FDR-CE results for GPT-4o with varying parameters and scoring functions. We use $\varepsilon_S = 0.3$, $\delta_S = 0.1$, $\alpha = 0.35$, and $\varepsilon_E = 0.05$ for Figure 3(a) and 3(b) and $\varepsilon_S = 0.2$, $\delta_S = 0.1$, $\alpha = 0.15$, and $\varepsilon_E = 0.05$ for Figure 3(c)

Table 2. Comparison results of before and after applying SCG on various code generators, including CODET(Chen et al., 2023a), LDB(Zhong et al., 2024), and SFS(Light et al., 2025) with GPT-3.5-Turbo. We use $\alpha = 0.3$, $\delta_S = 0.1$, $\varepsilon_E = 0.05$, and $\varepsilon_S = 0.25$ for all datasets. We use the mixed scoring function f_{mix} for this experiment. For HumanEval-f, we utilized a full dataset, whereas for MBPP-f, we constructed a new dataset using the union of subsets released by the authors of baseline methods. The FDR-CE satisfying the desired guarantees are marked in **bold**.

Methods	Base Model		Chen et al. (2023a)		Zhong et al. (2024)		Light et al. (2025)	
	w/o SCG	w/ SCG	w/o SCG	w/ SCG	w/o SCG	w/ SCG	w/o SCG	w/ SCG
MBPP-F								
1 - PASS@1 ($\downarrow$)	0.496 $\pm$ 0.039	0.163 $\pm$ 0.058	0.502 $\pm$ 0.034	0.155 $\pm$ 0.046	0.442 $\pm$ 0.044	0.150 $\pm$ 0.045	0.486 $\pm$ 0.037	0.151 $\pm$ 0.043
FDR-CE ($\downarrow$)	0.491 $\pm$ 0.042	0.145$\pm$0.052	0.498 $\pm$ 0.034	0.148$\pm$0.045	0.442 $\pm$ 0.044	0.142$\pm$0.043	0.483 $\pm$ 0.039	0.140$\pm$0.042
EFFICIENCY ($\uparrow$)	1.000 $\pm$ 0.000	0.576 $\pm$ 0.039	1.000 $\pm$ 0.000	0.587 $\pm$ 0.040	1.000 $\pm$ 0.000	0.655 $\pm$ 0.044	1.000 $\pm$ 0.000	0.604 $\pm$ 0.038
HUMAN-EVAL-F								
1 - PASS@1 ($\downarrow$)	0.294 $\pm$ 0.057	0.100 $\pm$ 0.069	0.308 $\pm$ 0.058	0.129 $\pm$ 0.072	0.253 $\pm$ 0.069	0.143 $\pm$ 0.071	0.274 $\pm$ 0.068	0.135 $\pm$ 0.059
FDR-CE ($\downarrow$)	0.305 $\pm$ 0.055	0.112$\pm$0.075	0.312 $\pm$ 0.062	0.135$\pm$0.069	0.230 $\pm$ 0.062	0.121$\pm$0.066	0.288 $\pm$ 0.072	0.158$\pm$0.058
EFFICIENCY ($\uparrow$)	1.000 $\pm$ 0.000	0.778 $\pm$ 0.065	1.000 $\pm$ 0.000	0.808 $\pm$ 0.061	1.000 $\pm$ 0.000	0.878 $\pm$ 0.052	1.000 $\pm$ 0.000	0.850 $\pm$ 0.056

5.2.1. CONTROLLABILITY AND SELECTION EFFICIENCY

Figure 2 shows that **SCG** controls the FDR-CE. To this end, we conducted random experiments. We ran the experiment 50 times by randomly splitting the calibration set and test set at 8:2 ratio each time. The whiskers on each plot denote the range between δ_S and $1 - \delta_S$ percentile of the distributions.

SCG-manual may perform better than our method depending on the choice of k . However, manually selecting an appropriate k for different situations is a challenging task. **SCG-small** bounds the FDR-CE successfully. However, this method demonstrates lower efficiency compared to our method. The result stems from a lack of unit tests to correctly infer *expected functional correctness*, illustrating the advantage of fuzzing in learning. **SCG-H** shows that it is not able to bound a desired FDR-CE, as it ignores an estimation error for inferring expected functional correctness.

Our method shows how it successfully bounds desired FDR-CEs on diverse models (Figure 2), diverse parameters (Figure 3, Figure 5), diverse datasets (Table 1), diverse methods (Table 2), and diverse programming languages (Table 6). In Figure 2, 3, and 5, this is shown by the upper whisker bar lying below the desired FDR-CE in dotted line, while Table 1, 2, 6, the results are indicated by bold text. Table 3 shows qualitative results of our method that accepts correct code and rejects uncertain code. Note that a poorly perform-

ing model, *e.g.*, CodeLlama in Figure 4(a), may not find a selective generator with a desired FDR-CE ε_S . This is an expected result due to an un-calibrated scoring function (Lee et al., 2024), as discussed in Section 5.2.4.

5.2.2. BENEFIT OF FUZZING

We show the benefits of fuzzing in learning and evaluation. We empirically show the benefit of fuzzing in learning. As shown in Figure 5(b), the FDR-CE bound (at the top of the whisker) gets tighter to a desired FDR-CE level *without violating it* because smaller ε_E requires a larger number of unit tests, thus providing a precise estimation on expected functional correctness. This shows that generated unit tests by fuzzing help to provide a tighter FDR-CE guarantee, with higher selection efficiency.

Additionally, we demonstrate that automatic unit test generation is beneficial in rigorous evaluation. As shown in Figure 5(c), the FDR-CE decreases as the number of unit tests for evaluation increases. Recalling that α -entailment is determined by comparing the lower bound of expected functional correctness with $1 - \alpha$ as in Definition 4.1, the lower bound gets tighter as we use more unit tests to evaluate code. The tighter lower bound results in more accurate comparison and evaluation, thus reducing the FDR-CE. This shows that fuzzing has a benefit of reducing evaluation error.

5.2.3. EXTENSION OVER PRIOR WORK

We compare our method **SCG** with prior code generation baselines and demonstrate that these baselines can be extended through our approach to provide additional functionality guarantees. As further discussed in Section 1.1, improvements in functionality – inherently reducing *functional hallucination* – can be broadly categorized into two approaches: (1) enhancing the model performance during training or fine-tuning (OpenAI et al., 2025; Guo et al., 2025) or (2) employing post-filtering methods (Chen et al., 2023a; Zhong et al., 2024; Light et al., 2025). In this work, we focus on comparison with post-filtering methods, as **SCG** post-processes the generated code to control hallucinations but training and fine-tuning based methods are orthogonal to this work.

Table 2 compares **SCG** with prior post-filtering methods. In particular, **SCG** controls and achieves lower FDR-CE than prior methods depending on the choice of ε_S . Furthermore, Table 2, 7, and Figure 3(a) demonstrate the applicability of our method in a model- and method-agnostic manner across diverse experiment setups, including integration with finetuning methods (e.g., DeepSeek GRPO, OpenAI RLHF) and compatibility with post-processing methods.

5.2.4. EFFECT OF SCORING FUNCTION

We demonstrate the effect of calibration on our method **SCG**. As shown in Figure 3(c), the choice of scoring function affects whether FDR-CE can be successfully bounded to the desired ε_S . In particular, f_{verb} in Figure 3(c) fails to properly bound the FDR-CE. Furthermore, CodeLlama in Figure 4(a) fails to find a selective generator with a desired ε_S , due to an un-calibrated scoring function (Lee et al., 2024). Thus, the model finds a minimum FDR-CE by returning $\hat{U}$ in this case. These results underscore the importance of selecting appropriate scoring functions for the efficacy of our method.

6. Conclusion

This paper considers the code hallucination problem. In particular, we define the concept of code entailment based on automatically generated unit tests via fuzzing, one of the dynamic code analysis tools. Given this, we propose a learning algorithm for selective code generators to theoretically control the hallucination in the FDR of selective generators. We further leverage fuzzing to automatically generate unit tests for learning and evaluation purposes, enabling the large-scale collection of unit tests. Lastly, we demonstrate the controllability of the proposed selective generator and its selection efficiency over open and closed code generators under different experiment setups.

Limitations. The proposed method controls the rate of hallucination, but its selection efficiency heavily depends on the quality of code generation models, requiring improve-

ment for code generators. Moreover, the i.i.d. assumption for the FDR controllability guarantee limits its applicability in distribution-shifting environments.

Impact Statement

This paper presents work whose goal is to advance the field of Machine Learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

Acknowledgment

We appreciate constructive feedback from anonymous reviewers. This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) and the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2019-II191906, Artificial Intelligence Graduate School Program (POSTECH) (10%); RS-2024-00457882, National AI Research Lab Project (25%); No. RS-2024-00509258 and No. RS-2024-00469482, Global AI Frontier Lab (25%); RS-2025-00560062 (40%)).

References

- Ahn, J., Verma, R., Lou, R., Liu, D., Zhang, R., and Yin, W. Large language models for mathematical reasoning: Progresses and challenges. In Falk, N., Papi, S., and Zhang, M. (eds.), *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: Student Research Workshop*, pp. 225–237, St. Julian’s, Malta, March 2024. Association for Computational Linguistics. URL <https://aclanthology.org/2024.eacl-srw.17/>.
- Alshahwan, N., Chheda, J., Finogenova, A., Gokkaya, B., Harman, M., Harper, I., Marginean, A., Sengupta, S., and Wang, E. Automated unit test improvement using large language models at meta. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering, FSE 2024*, pp. 185–196, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400706585. doi: 10.1145/3663529.3663839. URL <https://doi.org/10.1145/3663529.3663839>.
- Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., and Sutton, C. Program synthesis with large language models, 2021. URL <https://arxiv.org/abs/2108.07732>.
- Bowman, S., Angeli, G., Potts, C., and Manning, C. D. A large annotated corpus for learning natural language

- inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 632–642, 2015.
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners, 2020.
- Cassano, F., Gouwar, J., Nguyen, D., Nguyen, S., Phipps-Costin, L., Pinckney, D., Yee, M.-H., Zi, Y., Anderson, C. J., Feldman, M. Q., et al. Multipl-e: A scalable and extensible approach to benchmarking neural code generation. *arXiv preprint arXiv:2208.08227*, 2022.
- Chen, B., Zhang, F., Nguyen, A., Zan, D., Lin, Z., Lou, J.-G., and Chen, W. Codet: Code generation with generated tests. In *The Eleventh International Conference on Learning Representations*, 2023a. URL <https://openreview.net/forum?id=ktrw68Cmu9c>.
- Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., and Zaremba, W. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- Chen, W., Ma, X., Wang, X., and Cohen, W. W. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *Transactions on Machine Learning Research*, 2023b.
- Chen, Y. and Su, Z. Guided differential testing of certificate validation in ssl/tls implementations. In *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering, ESEC/FSE 2015*, pp. 793–804, New York, NY, USA, 2015. Association for Computing Machinery. ISBN 9781450336758. doi: 10.1145/2786805.2786835. URL <https://doi.org/10.1145/2786805.2786835>.
- Claessen, K. and Hughes, J. Quickcheck: a lightweight tool for random testing of haskell programs. *SIGPLAN Not.*, 35(9):268–279, September 2000. ISSN 0362-1340. doi: 10.1145/357766.351266. URL <https://doi.org/10.1145/357766.351266>.
- Clopper, C. J. and Pearson, E. S. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934.
- Ding, Y., Peng, J., Min, M. J., Kaiser, G., Yang, J., and Ray, B. Semcoder: Training code language models with comprehensive semantics reasoning. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 60275–60308. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/6efcc7fd8efeee29a050a79c843c90e0-Paper-Conference.pdf.
- Du, M., Tuan, L. A., Ji, B., Liu, Q., and Ng, S.-K. Mercury: A code efficiency benchmark for code large language models. In Globerson, A., Mackey, L., Belgrave, D., Fan, A., Paquet, U., Tomczak, J., and Zhang, C. (eds.), *Advances in Neural Information Processing Systems*, volume 37, pp. 16601–16622. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/1df1df43b58845650b8dada00fca9772-Paper-Datasets_and_Benchmarks_Track.pdf.
- Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Musuvathi, M., and Lahiri, S. Exploring the effectiveness of llm based test-driven interactive code generation: User study and empirical evaluation. In *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, ICSE-Companion '24*, pp. 390–391, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400705021. doi: 10.1145/3639478.3643525. URL <https://doi.org/10.1145/3639478.3643525>.
- Fakhoury, S., Kuppe, M., Lahiri, S. K., Ramanand, T., and Swamy, N. 3dgen: Ai-assisted generation of provably correct binary format parsers. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pp. 2535–2547, Los Alamitos, CA, USA, May 2025. IEEE Computer Society. doi: 10.1109/ICSE55347.2025.00173. URL <https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00173>.
- Felsing, D., Grebing, S., Klebanov, V., Rümmer, P., and Ulbrich, M. Automating regression verification. In *Proceedings of the 29th ACM/IEEE International Conference on Automated Software Engineering, ASE '14*, pp. 349–360, New York, NY, USA, 2014. Association

- for Computing Machinery. ISBN 9781450330138. doi: 10.1145/2642937.2642987. URL <https://doi.org/10.1145/2642937.2642987>.
- Geifman, Y. and El-Yaniv, R. Selective classification for deep neural networks. *Advances in neural information processing systems*, 30, 2017.
- Google. Atheris: A coverage-guided, native python fuzzer, 2020. URL <https://github.com/google/atheris>.
- Guo, D., Yang, D., Zhang, H., Song, J., Wang, P., Zhu, Q., Xu, R., Zhang, R., Ma, S., Bi, X., et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- Hendrycks, D., Basart, S., Kadavath, S., Mazeika, M., Arora, A., Guo, E., Burns, C., Puranik, S., He, H., Song, D., and Steinhardt, J. Measuring coding challenge competence with APPS. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL <https://openreview.net/forum?id=sD93GOzH3i5>.
- Hossain, S. B., Jiang, N., Zhou, Q., Li, X., Chiang, W.-H., Lyu, Y., Nguyen, H., and Tripp, O. A deep dive into large language models for automated bug localization and repair. *Proc. ACM Softw. Eng.*, 1(FSE), July 2024. doi: 10.1145/3660773. URL <https://doi.org/10.1145/3660773>.
- Huang, B., Lu, S., Wan, X., and Duan, N. Enhancing large language models in coding through multi-perspective self-consistency. In Ku, L.-W., Martins, A., and Srikumar, V. (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1429–1450, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.78. URL <https://aclanthology.org/2024.acl-long.78/>.
- Jakobs, M.-C. and Wiesner, M. Peqtest: Testing functional equivalence. In Johnsen, E. B. and Wimmer, M. (eds.), *Fundamental Approaches to Software Engineering*, pp. 184–204, Cham, 2022. Springer International Publishing. ISBN 978-3-030-99429-7.
- Kim, T., Han, H., Park, S., Jeong, D. R., Kim, D., Kim, D., Kim, E., Kim, J., Wang, J., Kim, K., Ji, S., Song, W., Zhao, H., Chin, A., Lee, G., Stevens, K., Alharthi, M., Zhai, Y., Zhang, C., Jang, J., Jang, Y., Askar, A., Kim, D., Fleischer, F., Cho, J., Kim, J., Ko, K., Yun, I., Park, S., Baik, D., Lee, H., Heo, H., Gwon, M., Lee, M., Baek, M., Min, S., Kim, W., Jin, Y., Park, Y., Choi, Y., Jung, J., Lee, G., Jang, J., Kim, K., Cha, Y., and Kim, Y. Atlantis: Ai-driven threat localization, analysis, and triage intelligence system, 2025. URL <https://arxiv.org/abs/2509.14589>.
- Kuhn, L., Gal, Y., and Farquhar, S. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=VD-AYtP0dve>.
- Lahiri, S. K., Hawblitzel, C., Kawaguchi, M., and Rebêlo, H. Symdiff: A language-agnostic semantic diff tool for imperative programs. In Madhusudan, P. and Seshia, S. A. (eds.), *Computer Aided Verification*, pp. 712–717, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg. ISBN 978-3-642-31424-7.
- Le, H., Wang, Y., Gotmare, A. D., Savarese, S., and Hoi, S. CodeRL: Mastering code generation through pretrained models and deep reinforcement learning. In Oh, A. H., Agarwal, A., Belgrave, D., and Cho, K. (eds.), *Advances in Neural Information Processing Systems*, 2022. URL <https://openreview.net/forum?id=WaGvb7OzySA>.
- Lee, M., Kim, K., Kim, T., and Park, S. Selective generation for controllable language models. *Advances in Neural Information Processing Systems*, 37:50494–50527, 2024.
- Li, D., Cao, S., Cao, C., Li, X., Tan, S., Keutzer, K., Xing, J., Gonzalez, J. E., and Stoica, I. S*: Test time scaling for code generation. In Christodoulopoulos, C., Chakraborty, T., Rose, C., and Peng, V. (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 15964–15978, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.865. URL <https://aclanthology.org/2025.findings-emnlp.865/>.
- Li, Y., Choi, D., Chung, J., Kushman, N., Schrittwieser, J., Leblond, R., Eccles, T., Keeling, J., Gimeno, F., Lago, A. D., Hubert, T., Choy, P., de Masson d’Autume, C., Babuschkin, I., Chen, X., Huang, P.-S., Welbl, J., Goyal, S., Cherepanov, A., Molloy, J., Mankowitz, D. J., Robson, E. S., Kohli, P., de Freitas, N., Kavukcuoglu, K., and Vinyals, O. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158. URL <https://www.science.org/doi/abs/10.1126/science.abq1158>.
- Light, J., Wu, Y., Sun, Y., Yu, W., Liu, Y., Zhao, X., Hu, Z., Chen, H., and Cheng, W. SFS: Smarter code space search improves LLM inference scaling. In *The Thirteenth International Conference on Learning Representations*, 2025.

- URL <https://openreview.net/forum?id=MCHuGOkExF>.
- Liu, J., Xia, C. S., Wang, Y., and Zhang, L. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. In *Proceedings of the 37th International Conference on Neural Information Processing Systems, NIPS '23*, Red Hook, NY, USA, 2023. Curran Associates Inc.
- Manakul, P., Liusie, A., and Gales, M. Selfcheckgpt: Zero-resource black-box hallucination detection for generative large language models. In *The 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- Massalin, H. Superoptimizer: a look at the smallest program. In *Proceedings of the Second International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS II*, pp. 122–126, New York, NY, USA, 1987. Association for Computing Machinery. ISBN 0818608056. doi: 10.1145/36206.36194. URL <https://doi.org/10.1145/36206.36194>.
- McKeeman, W. M. Differential testing for software. *Digit. Tech. J.*, 10:100–107, 1998.
- Miller, B. P., Fredriksen, L., and So, B. An empirical study of the reliability of unix utilities. *Communications of the ACM*, 33(12):32–44, 1990.
- Mohri, C. and Hashimoto, T. Language models with conformal factuality guarantees. In *Proceedings of the 41st International Conference on Machine Learning, ICML'24*. JMLR.org, 2024.
- OpenAI. Gpt-4 technical report, 2024. URL <https://arxiv.org/abs/2303.08774>.
- OpenAI. Introducing gpt-4.1 in the api, 2025. URL <https://openai.com/index/gpt-4-1/>.
- OpenAI, :, El-Kishky, A., Wei, A., Saraiva, A., Minaiev, B., Selsam, D., Dohan, D., Song, F., Lightman, H., Clavera, I., Pachocki, J., Tworek, J., Kuhn, L., Kaiser, L., Chen, M., Schwarzer, M., Rohaninejad, M., McAleese, N., o3 contributors, Mürk, O., Garg, R., Shu, R., Sidor, S., Kosaraju, V., and Zhou, W. Competitive programming with large reasoning models, 2025. URL <https://arxiv.org/abs/2502.06807>.
- Quach, V., Fisch, A., Schuster, T., Yala, A., Sohn, J. H., Jaakkola, T. S., and Barzilay, R. Conformal language modeling. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=pzUhfQ74c5>.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X. E., Adi, Y., Liu, J., Sauvestre, R., Remez, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C. C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., and Synnaeve, G. Code llama: Open foundation models for code, 2024. URL <https://arxiv.org/abs/2308.12950>.
- Semmler. Codeql, 2019. URL <https://codeql.github.com/>.
- Spiess, C., Gros, D., Pai, K. S., Pradel, M., Rabin, M. R. I., Alipour, A., Jha, S., Devanbu, P., and Ahmed, T. Calibration and correctness of language models for code. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering, ICSE '25*, pp. 540–552. IEEE Press, 2025. ISBN 9798331505691. doi: 10.1109/ICSE55347.2025.00040. URL <https://doi.org/10.1109/ICSE55347.2025.00040>.
- Team, G., Georgiev, P., Lei, V. I., Burnell, R., Bai, L., Gulati, A., Tanzer, G., Vincent, D., Pan, Z., Wang, S., et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context, 2024. URL <https://arxiv.org/abs/2403.05530>.
- Tian, K., Mitchell, E., Zhou, A., Sharma, A., Rafailov, R., Yao, H., Finn, C., and Manning, C. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback. In Bouamor, H., Pino, J., and Bali, K. (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 5433–5442, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.330. URL <https://aclanthology.org/2023.emnlp-main.330/>.
- Tian, Y., Yan, W., Yang, Q., Zhao, X., Chen, Q., Wang, W., Luo, Z., Ma, L., and Song, D. Codehalu: Investigating code hallucinations in llms via execution-based verification. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 25300–25308, 2025.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models, 2023.
- Vovk, V., Gammerman, A., and Shafer, G. *Algorithmic learning in a random world*. Springer Science & Business Media, 2005.
- Williams, A., Nangia, N., and Bowman, S. R. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of NAACL-HLT*, pp. 1112–1122, 2018.

Zalewski, M. Technical “whitepaper” for afl-fuzz. *URL:* http://lcamtuf.coredump.cx/afl/technical_details.txt, 2014.

Zhang, Z., Wang, Y., Wang, C., Chen, J., and Zheng, Z. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation, 2025. URL <https://arxiv.org/abs/2409.20550>.

Zhong, L., Wang, Z., and Shang, J. Debug like a human: A large language model debugger via verifying runtime execution step by step. In *Findings of the Association for Computational Linguistics ACL 2024*, pp. 851–870, 2024.

A. Preliminary

Textual Entailment. In natural languages, textual entailment is a concept of evaluating a semantic relation between two sentences by checking an entailment relation (Bowman et al., 2015). In particular, denoting two sentences by a premise and a hypothesis, we say that a premise entails a hypothesis if the hypothesis is true given the premise. Otherwise, we say the premise contradicts the hypothesis.

This textual entailment has been used to measure semantic correctness between a question and an answer in learning language models (Mohri & Hashimoto, 2024; Lee et al., 2024). If a generated answer entails a true one, we can consider the generated answer as true in textual entailment. In evaluating correctness of generated answers in natural language processing, introducing textual entailment is crucial as a simple, traditional exact match, *i.e.*, the generated answer and the true one are exactly the same, does not measure the semantic relation between two answers. However, in code generation, there is no notion of entailment to check the semantic correctness between two code snippets. We overcome this hurdle by introducing *code entailment* for measuring functional correctness, leveraging executable properties of code.

Dynamic Code Analysis via Fuzzing. Computer programs suffer from undefined behaviors, called *bugs*, *e.g.*, crash by buffer overflow. To find the bugs, security researchers have extensively developed static and dynamic code analysis tools, *e.g.*, CodeQL (Semmler, 2019), AFL (Zalewski, 2014), and Atheris (Google, 2020).

The dynamic analysis tools exploit the executable property of code to find bugs, while the static analysis tools inspect code without execution. We mainly focus on more informative dynamic analysis tools. In particular, fuzzing, a representative class of methods for dynamic code analysis, generates the input of a given program, called seed, executes the program with the input, and checks whether undefined behaviors can be observable. Given the observation, fuzzing methods randomly mutate the input of programs to explore wider execution paths or exploit execution paths toward targeted code. In this paper, we re-purpose fuzzing methods for identifying functionality of code, instead of finding bugs in code by generating the input and output pairs of code.

B. Qualitative Result of our Methods

Table 3. Qualitative results of our selective code generator with GPT-4o ($\epsilon_S = 0.3$, $\delta_S = 0.1$, $\epsilon_E = 0.05$, and $\alpha = 0.15$). The accepted generated code is α -entailment with a score of -0.05 , meaning high certainty, where the code is also semantically correct. The rejected code is not α -entailment with the score of -53.87 , meaning low certainty probably due to the complexity of the question. In particular, it returns a wrong answer, 2, for an input, [1,1,0,1,0,0,0,1,0,1,0,0,1,1,0,1], where the expected answer is 14, due to the inequality in “count > max_count” that does not follow the tie instruction. Note that the contents are manually truncated for presentation purposes.

Question	Canonical Solution	Generated Code	Selection (ours)
You are given an array to sort but you’re expected to start sorting from a specific position of the array (in ascending order) and optionally you’re given the number of items to sort.	<pre>def sect_sort(lst, start, length=0): end = start + length \ if length else \ len(lst) return lst[start] + \ sorted(lst[start:end]) + \ lst[end:]</pre>	<pre>def sect_sort(array, start, length=0): if length == 0: end = len(array) else: end = start + length array[start:end] = \ sorted(array[start:end]) return array</pre>	Accept
Given an array containing only zeros and ones, find the index of the zero that, if converted to one, will make the longest sequence of ones. ... Return the last index if there are multiple results.	<pre>def replace_zero(arr): m, im, i, lst = 0, -1, -1, (''.join(map(str, arr)).split('0')) for a, b in zip(lst, lst[1:]): i += len(a) + 1 candidate = len(a) + len(b) + 1 if m <= candidate: im, m = i, candidate return im</pre>	<pre>def replace_zero(arr): max_count = 0 max_index = -1 count = 0 last_zero_idx = -1 consecutive_count_before_zero = 0 for i in range(len(arr)): if arr[i] == 1: count += 1 else: count = i - last_zero_idx last_zero_idx = i if count > max_count: max_count = count max_index = last_zero_idx return max_index</pre>	Reject

C. Algorithm

Algorithm 1 Unit test size computation.

```

1: procedure COMPUNITTESTSIZE( $\alpha, \varepsilon_E, \mathcal{F}(\mathbf{x}), \bar{\mathbf{y}}, n_{\max}$ )
2:  $n_{\mathbf{x}} \leftarrow 0$ 
3: while  $\hat{L}(\mathcal{F}(\mathbf{x}), \bar{\mathbf{y}}, n_{\mathbf{x}}, \varepsilon_E) < 1 - \alpha$  do
4:    $(\mathbf{u}, \mathbf{v}) \sim \mathcal{F}(\mathbf{x})$ 
5:    $n_{\mathbf{x}} \leftarrow n_{\mathbf{x}} + 1$ 
6:   if  $n_{\mathbf{x}} \geq n_{\max}$  then
7:     break ▷ break for infeasible cases
8:   end if
9: end while
10: return  $n_{\mathbf{x}}$ 
    
```

Algorithm 2 Selective Code Generator Learning

```

1: procedure LEARNSCG( $f, \mathbf{Z}, G, \alpha, \varepsilon_E, \delta_S$ )
2:  $\mathbf{Z}' \leftarrow \text{SORT}_f(\mathbf{Z})$  ▷ Increasing order of  $f(\mathbf{x}_i, G(\mathbf{x}_i))$ 
3:  $(\underline{i}, \bar{i}) \leftarrow (1, |\mathbf{Z}|)$ 
4: for  $i = 1$  to  $\lceil \log_2 |\mathbf{Z}| \rceil$  do
5:    $i_{\text{mid}} \leftarrow \lceil \frac{\underline{i} + \bar{i}}{2} \rceil$ 
6:    $\tau_S^{(i)} \leftarrow f(\mathbf{x}_{i_{\text{mid}}}, G(\mathbf{x}_{i_{\text{mid}}}))$  ▷ Choose a candidate  $\tau$ 
7:    $\hat{\mathbf{Z}}^{(i)} \leftarrow \{(\mathbf{x}, -) \in \mathbf{Z}' \mid f(\mathbf{x}, G(\mathbf{x})) \geq \tau_S^{(i)}\}$  ▷ Build a selected calibration set by  $\tau_S^{(i)}$ 
8:    $\hat{k}^{(i)} \leftarrow \sum_{(\mathbf{x}, \mathbf{y}, -) \in \hat{\mathbf{Z}}^{(i)}} \mathbb{1}(G(\mathbf{x}) \notin \hat{E}_{\alpha, \varepsilon_E}(\mathbf{x}))$  ▷ Count false discoveries.
9:    $\hat{U}^{(i)} \leftarrow \hat{U}_{\text{Binom}}(\hat{k}^{(i)}, |\hat{\mathbf{Z}}^{(i)}|, \delta_S / \lceil \log_2 |\mathbf{Z}| \rceil)$  ▷ Bound the FDR-CE.
10:  if  $\varepsilon_E + \hat{U}^{(i)} \leq \varepsilon_S$  then
11:     $\bar{i} \leftarrow i_{\text{mid}}$  ▷ Keep  $\tau_S^{(i)}$  that controls a desired FDR-CE.
12:  else
13:     $\underline{i} \leftarrow i_{\text{mid}}$ 
14:  end if
15: end for
16: return  $\tau_S^{(i)}, \varepsilon_E + \hat{U}^{(i)}$ 
    
```

D. SCG-small Detail

This is our method but only using unit tests, provided by the APPS dataset to show the efficacy of generated unit tests via fuzzing. In particular, the APPS dataset provides an average of 21 unit tests per problem. However, the number of unit tests varies across the APPS dataset, and some problems lack unit tests. As it is difficult to directly apply our method to the original dataset, we sampled 21 test cases for each problem from generated unit tests via fuzzing to apply our algorithm. To support the validity of this baseline method, we provide additional experiments in Appendix L.5 regarding the quality of unit tests. Lastly, it is worth noting that this baseline is only applicable to the APPS dataset, due to an insufficient number of unit tests in other datasets to determine code entailment and obtain meaningful results.

E. Scoring Functions

Here, we denote p_i as the probability assigned by a code generator G to generate the i -th token.

- **Length-normalized log-probability** f_{norm} : This method collects the log-probability each token used to generate code then normalizes by the number of tokens, *i.e.*, $f_{\text{norm}}(\mathbf{x}, G(\mathbf{x})) = \frac{\sum_i \ln p_i}{|G(\mathbf{x})|}$. We use f_{norm} as the default scoring function unless specified.
- **Lowest log-probability** f_{min} : This method collects the log-probability of each generated token then selects the lowest value, *i.e.*, $f_{\text{min}}(\mathbf{x}, G(\mathbf{x})) = \min_i \ln p_i$.
- **Sequence log-probability** f_{seq} : This method collects the log-probability of each token to calculate the probability of generated code, *i.e.*, $f_{\text{seq}}(\mathbf{x}, G(\mathbf{x})) = \sum_i \ln p_i$.
- **Verbalized probability** (Tian et al., 2023), (Spiess et al., 2025) f_{verb} : The model used for code generation is prompted to produce a confidence value, with the prompt $\mathbf{p}_{\text{conf}}(G(x))$, *i.e.*, $f_{\text{verb}}(x, G(x)) = G(\mathbf{p}_{\text{conf}}(G(x)))$.
- **Mixed Scoring Function** We introduce a new scoring function that combines the length-normalized log probability and the pass rate of additional U unit tests, *i.e.*, $f_{\text{mixed}}(\mathbf{x}, G(\mathbf{x})) := 0.5f_{\text{norm}}(\mathbf{x}, G(\mathbf{x})) + 0.5 \frac{\sum_{(u_i, v_i) \sim \mathcal{F}(\mathbf{x})} \mathbb{1}(G(\mathbf{x})(u_i) = v_i)}{U}$

F. Datasets and Models

We use four datasets, APPS(Hendrycks et al., 2021), MBPP(Austin et al., 2021), HumanEval(Chen et al., 2021), and Mercury(Du et al., 2024), for calibration and evaluation. We use five large language models (LLMs), *GPT-4o*, *GPT-4.1*, *Gemini-1.5 Pro*, *CodeLlama 13B Instruct*, and *DeepSeek-R1* for code generation.

To determine code entailment of the generated code, we specifically selected datasets that provide a solution to the problem. For each problem, unit tests were generated using a dynamic analysis tool. The following Table 4 shows the result of the dynamic code analysis for each of the datasets.

To enhance the efficiency of execution path exploration, we manually post-processed each solution on the datasets for compatibility with the dynamic analysis tool. Although manual post-processing is not a mandatory step, it results in additional computational time and overhead for dynamic code analysis. During this step, we also excluded problems either lacking a solution or containing errors.

Table 4. The result of dynamic analysis for each dataset. We excluded problems that had errors in the original problem or with solutions that were difficult to analyze dynamically.

Dataset	Original Dataset	Solution-problem Error	Dynamic Analysis Error (<i>e.g.</i> , Atheris timeout)
APPS	10,000	1,274	307
MBPP	974	-	35
HumanEval	164	-	1
Mercury	1,889	-	279

G. Proof of Lemma 4.2

Recall that $e := \mathbb{1}(G(\mathbf{x}) \in E_\alpha(\mathbf{x}))$, $\hat{e} := \mathbb{1}(G(\mathbf{x}) \in \hat{E}_{\alpha, \varepsilon_E}(\mathbf{x}))$, where e denotes whether $\mathbf{y}$ α -entails $G(\mathbf{x})$ and $\hat{e}$ denotes the predicted result of α -entailment.

Additionally, we have the following due to Lemma 2.1:

$$\begin{aligned}\mathcal{R}_\alpha(\hat{S}) &= \mathbb{P}\{e = 0, \hat{e} = 1 \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\} - \mathbb{P}\{e = 1, \hat{e} = 0 \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\} + \mathcal{R}_{\alpha, \varepsilon_E}(\hat{S}) \\ &\leq \mathbb{P}\{e = 0, \hat{e} = 1 \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\} + \mathcal{R}_{\alpha, \varepsilon_E}(\hat{S}),\end{aligned}$$

where the probability is taken over $\mathbf{x}$, $n_{\mathbf{x}}$, and $\mathbf{U}_{\mathbf{x}}$.

We, then, bound the first term $\mathbb{P}\{e = 0, \hat{e} = 1 \mid \hat{S}(\mathbf{x}) \neq \text{IDK}\}$. In particular, suppose that $\mathbf{x}$, and $n_{\mathbf{x}}$ which satisfies $e = 0$ and $\hat{S}(\mathbf{x}) \neq \text{IDK}$ are given. Here, $n_{\mathbf{x}}$ is determined using Algorithm 1 and recall that $\mathbf{U}_{\mathbf{x}} \sim \mathcal{F}(\mathbf{x})^{n_{\mathbf{x}}}$ denoting a set of the $n_{\mathbf{x}}$ number of input-output pairs from a unit test generator $\mathcal{F}$.

Then, we have the following:

$$\begin{aligned}\mathbb{P}_{\mathbf{U}_{\mathbf{x}}} \left\{ \hat{e} = 1 \mid e = 0, \hat{S}(\mathbf{x}) \neq \text{IDK} \right\} &= \mathbb{P}_{\mathbf{U}_{\mathbf{x}}} \left\{ G(\mathbf{x}) \in \hat{E}_{\alpha, \varepsilon_E}(\mathbf{x}) \mid G(\mathbf{x}) \notin E_\alpha(\mathbf{x}) \right\} \\ &= \mathbb{P}_{\mathbf{U}_{\mathbf{x}}} \left\{ \hat{L}(\mathcal{F}(\mathbf{x}), G(\mathbf{x}), n_{\mathbf{x}}, \varepsilon_E) \geq 1 - \alpha \mid \mathbb{P}\{G(\mathbf{x})(\mathbf{u}) = \mathbf{v}\} < 1 - \alpha \right\} \\ &\leq \mathbb{P}_{\mathbf{U}_{\mathbf{x}}} \left\{ \hat{L}(\mathcal{F}(\mathbf{x}), G(\mathbf{x}), n_{\mathbf{x}}, \varepsilon_E) > \mathbb{P}\{G(\mathbf{x})(\mathbf{u}) = \mathbf{v}\} \mid \mathbb{P}_{\mathbf{y}}\{G(\mathbf{x})(\mathbf{u}) = \mathbf{v}\} < 1 - \alpha \right\} \\ &\leq \varepsilon_E,\end{aligned}$$

where the last inequality holds due to the confidence level of the binomial tail bound $\hat{L}$.

From this, letting F be $e = 0 \wedge \hat{S}(\mathbf{x}) \neq \text{IDK}$ ¹, we have the following:

$$\begin{aligned}\mathbb{P} \left\{ e = 0, \hat{e} = 1 \mid \hat{S}(\mathbf{x}) \neq \text{IDK} \right\} &= \mathbb{P} \left\{ \hat{e} = 1 \mid e = 0, \hat{S}(\mathbf{x}) \neq \text{IDK} \right\} \mathbb{P} \left\{ e = 0 \mid \hat{S}(\mathbf{x}) \neq \text{IDK} \right\} \\ &\leq \mathbb{P} \left\{ \hat{e} = 1 \mid e = 0, \hat{S}(\mathbf{x}) \neq \text{IDK} \right\} \\ &= \int \mathbb{1}\{\hat{e} = 1\} p(\mathbf{x}, n_{\mathbf{x}}, \mathbf{U}_{\mathbf{x}} \mid F) \, d\mathbf{x}, n_{\mathbf{x}}, \mathbf{U}_{\mathbf{x}} \\ &= \int \mathbb{1}\{\hat{e} = 1\} p(\mathbf{U}_{\mathbf{x}} \mid \mathbf{x}, n_{\mathbf{x}}, F) p(\mathbf{x}, n_{\mathbf{x}} \mid F) \, d\mathbf{x}, n_{\mathbf{x}}, \mathbf{U}_{\mathbf{x}} \\ &= \int \mathbb{P}_{\mathbf{U}_{\mathbf{x}}} \left\{ \hat{e} = 1 \mid e = 0, \hat{S}(\mathbf{x}) \neq \text{IDK} \right\} p(\mathbf{x}, n_{\mathbf{x}} \mid F) \, d\mathbf{x}, n_{\mathbf{x}} \\ &\leq \int \varepsilon_E \cdot p(\mathbf{x}, n_{\mathbf{x}} \mid F) \, d\mathbf{x}, n_{\mathbf{x}} \\ &= \varepsilon_E,\end{aligned}$$

as claimed.

H. A Proof of Theorem 4.3

We use the same proof techniques used in (Geifman & El-Yaniv, 2017) and (Lee et al., 2024). Here, we add the proof for completeness.

First, from Lemma 4.2 and the binomial tail bound, we have the following point bound for any $\hat{S}$, $\varepsilon_E \in (0, 1)$, and $\delta_S \in (0, 1)$:

$$\mathcal{R}_\alpha(\hat{S}) \leq \varepsilon_E + \hat{U}_{\text{Binom}}(\hat{k}; |\hat{\mathbf{Z}}|, \delta_S / \lceil \log_2 |\mathbf{Z}| \rceil) \quad (5)$$

with probability at least $1 - \delta_S / \lceil \log_2 |\mathbf{Z}| \rceil$, where the probability is taken over $\mathbf{Z} \sim \mathcal{D}^n$ with a fixed n . Here, $\hat{k}$, $\hat{\mathbf{Z}}$, and $\mathbf{Z}$ are as defined in Section 4.5, and $\hat{U}_{\text{Binom}}$ is the standard binomial tail upper bound.

Let $\mathcal{H}$ be a data-dependent set of selective generators, parameterized by τ , with a fixed size m , i.e., $|\mathcal{H}| = m$ (where $m = \lceil \log_2 n \rceil$ in our case) and $\mathcal{H}_{\varepsilon_S} := \{\hat{S} \in \mathcal{H} \mid \mathcal{R}_\alpha(\hat{S}) > \varepsilon_E + \hat{U}_{\text{Binom}}(\hat{k}; |\hat{\mathbf{Z}}|, \delta_S/m)\}$, which is also data-dependent.

¹Note that the probability of the event F is positive unless G is “always-correct” (i.e., $\mathbf{y} \alpha$ -entails $G(\mathbf{x})$ for all $\mathbf{x}$ and $\mathbf{y}$).

Recall that $\hat{\tau}$ and $\hat{U}$ are the output of our Algorithm 2. Then, we have the following:

$$\begin{aligned}
 & \mathbb{P}_{\mathbf{Z}} \left\{ \mathcal{R}_{\alpha}(\hat{S}) > \hat{U} \right\} \\
 & \leq \mathbb{P}_{\mathbf{Z}} \left\{ \exists \hat{S} \in \mathcal{H}_{\varepsilon_S}, \mathcal{R}_{\alpha}(\hat{S}) > \varepsilon_E + \hat{U}_{\text{Binom}}(\hat{k}; |\hat{\mathbf{Z}}|, \delta_S/m) \right\} \\
 & \leq \sum_{j=1}^m \mathbb{P}_{\mathbf{Z}} \left\{ \mathcal{R}_{\alpha}(\hat{S}_j) > \varepsilon_E + \hat{U}_{\text{Binom}}(\hat{k}; |\hat{\mathbf{Z}}_j|, \delta_S/m) \right\} \tag{6} \\
 & = \sum_{j=1}^m \sum_{i_j=0}^n \mathbb{P}_{\mathbf{Z}} \left\{ \mathcal{R}_{\alpha}(\hat{S}_j) > \varepsilon_E + \hat{U}_{\text{Binom}}(\hat{k}_j; |\hat{\mathbf{Z}}_j|, \delta_S/m), |\hat{\mathbf{Z}}_j| = i_j \right\} \\
 & = \sum_{j=1}^m \sum_{i_j=0}^n \mathbb{P}_{\mathbf{Z}} \left\{ \mathcal{R}_{\alpha}(\hat{S}_j) > \varepsilon_E + \hat{U}_{\text{Binom}}(\hat{k}_j; |\hat{\mathbf{Z}}_j|, \delta_S/m) \mid |\hat{\mathbf{Z}}_j| = i_j \right\} \mathbb{P}_{\mathbf{Z}} \left\{ |\hat{\mathbf{Z}}_j| = i_j \right\} \\
 & \leq \sum_{j=1}^m \sum_{i_j=0}^n \frac{\delta_S}{m} \mathbb{P}_{\mathbf{Z}} \left\{ |\hat{\mathbf{Z}}_j| = i_j \right\} \tag{7} \\
 & = \delta_S,
 \end{aligned}$$

where (6) holds due to a union bound and (7) satisfies due to the point bound in (5). This completes the proof.

I. Empirical Evaluation Metrics

We evaluate our method along with baselines based on the empirical FDR-CE, *i.e.*, $\widehat{\text{FDR-CE}}$, and empirical selection efficiency, *i.e.*, $\widehat{\text{SelEff}}$, where $\mathbf{Z}_t \sim \mathcal{D}^{n_{\text{test}}}$ is a test set, as follows:

$$\widehat{\text{FDR-CE}} := \frac{\sum_{(\mathbf{x}, \mathbf{y}, \cdot) \in \mathbf{Z}_t} \mathbb{1} \left(\hat{S}(\mathbf{x}) \notin \hat{E}_{\alpha, \varepsilon_E^t}(\mathbf{y}) \wedge \hat{S}(\mathbf{x}) \neq \text{IDK} \right)}{\sum_{(\mathbf{x}, \cdot) \in \mathbf{Z}_t} \mathbb{1} \left(\hat{S}(\mathbf{x}) \neq \text{IDK} \right)} \quad \text{and} \tag{8}$$

$$\widehat{\text{SelEff}} := \frac{1}{|\mathbf{Z}_t|} \sum_{(\mathbf{x}, \cdot) \in \mathbf{Z}_t} \mathbb{1}(\hat{S}(\mathbf{x}) \neq \text{IDK}). \tag{9}$$

We chose $\varepsilon_E^t = 0.01 \ll \varepsilon_E$ for estimating a true α -entailment set E_{α} .

J. Relationship between Pass@1 and FDR-CE

Given an instance $(\mathbf{x}, \mathbf{y})$ in the dataset, suppose there are $n_{\mathbf{x}}$ given unit tests provided. Here, $n_{\mathbf{x}}$ is the result of Algorithm 1. When evaluating a dataset with PASS@1, a generated code snippet is considered correct if it passes all $n_{\mathbf{x}}$ unit tests.

As $\alpha \rightarrow 0$, if $\mathbf{x}$ α -entails the generated code $G(\mathbf{x})$, then $n_{\mathbf{x}}$ must tend toward ∞ . Thus, $\mathcal{F}(\mathbf{x})$ exhibits *expected functional correctness* approaching 1. Therefore, with high probability, any finite set of $n_{\mathbf{x}}$ unit tests will be successfully executed without a failed test.

Conversely, if a generated code snippet successfully executes $n_{\mathbf{x}}$ unit tests as $n_{\mathbf{x}} \rightarrow \infty$, then with high probability, $\mathcal{F}(\mathbf{x})$ will α -entail the generated code $G(\mathbf{x})$.

Therefore, 1-PASS@1 and FDR-CE may be asymptotically equivalent as $\alpha \rightarrow 0$ and $n_{\mathbf{x}} \rightarrow \infty$.

The empirical result is presented in Table 1 and 6.

K. alpha-Entailment Code Example

Table 5. α -entailment example with $\alpha = 0.3$. The solution code accurately simulates the exact problem scenario, while the generated code simplifies the logic and therefore misses corner cases by missing the α fraction of unit tests generated from the solution code.

Question n people are standing in a line to play table tennis. At first, the first two players in the line play a game. Then the loser goes to the end of the line, and the winner plays with the next person from the line, and so on. They play until someone wins k games in a row. This player becomes the winner. For each of the participants, you know the power to play table tennis, and for all players these values are different. In a game the player with greater power always wins. Determine who will be the winner.
(a) Canonical Solution <pre> tmp = list(map(int, input().split())) n, k = tmp[0], tmp[1] a = list(map(int, input().split())) cur = a[0] w = 0 OK = False for i in range(1, 10000): op = a[i] if cur > op: a.append(op) w += 1 else: cur = op a.append(cur) w = 1 if w >= k: OK = True break if OK: print(cur) else: print(max(a)) </pre>
(b) Generated Solution <pre> def find_winner(n, k, powers): current_player_index = 0 current_wins = 0 max_power = powers[0] if k > n: print(max(powers)) return for i in range(1, n): if powers[current_player_index] > powers[i]: current_wins += 1 else: current_player_index = i current_wins = 1 if current_wins == k: print(powers[current_player_index]) return print(max_power) # Read input n, k = map(int, input().split()) powers = list(map(int, input().split())) find_winner(n, k, powers) </pre>

L. Additional Experiments

L.1. Varying Model

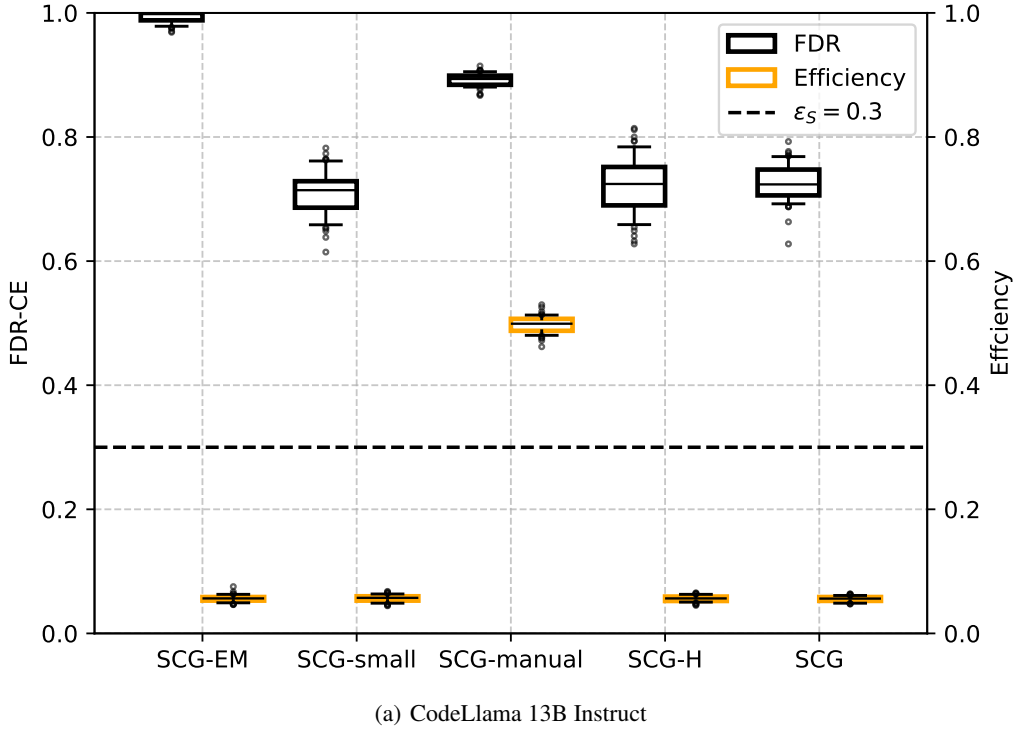


Figure 4. The box plots of the FDR-CE and selection efficiency for CodeLlama 13B Instruct. We set $\delta_S = 0.1$, $\varepsilon_S = 0.3$, $\varepsilon_E = 0.05$, and $\alpha = 0.35$. **SCG** fails to find a selective generator with a desired FDR-CE due to an uncalibrated scoring function.

L.2. Varying Parameter

We present FDR-CE and efficiency experiments with varying δ_S , varying ε_E , and varying number of unit tests in Figure 5.

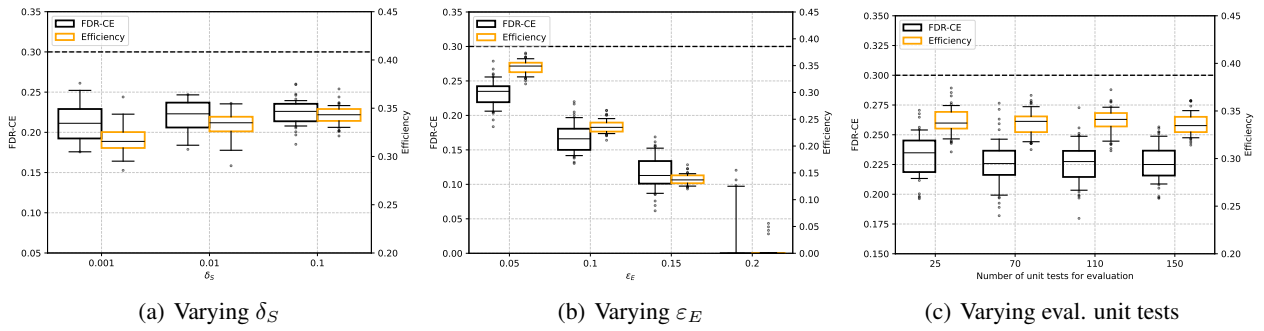


Figure 5. The FDR-CE results for GPT-4o with varying parameters ($\varepsilon_S = 0.3$, $\delta_S = 0.1$, $\alpha = 0.35$, and $\varepsilon_E = 0.05$). Each figure shows FDR-CE bound is satisfied for each setting. This is shown by the upper whisker bar lying below the desired FDR-CE in the dotted line. Figure 5(b) shows benefit of fuzzing in learning and Figure 5(c) shows the benefit of fuzzing in evaluation.

L.3. Various Programming Languages

Table 6. Comparison results of SCG against baseline methods on different programming languages with GPT-4.1 ($\alpha = 0.35, \delta_S = 0.1, \varepsilon_E = 0.05, \varepsilon_S = 0.3$ for all programming languages). We selected a subset of problems (5359 instances) from the APPS-f dataset that conform to the standard input/output format for executing unit tests. The FDR-CE satisfying the desired guarantees and the highest efficiency among methods that comply with the FDR-CE guarantees are marked in **bold**.

Methods	w/o Selective Generation	w/ Selective Generation				
	$\tau = -\infty$	SCG-EM	SCG-MANUAL	SCG-SMALL	SCG-H	SCG
CPP ($\varepsilon_S = 0.3$)						
1 - PASS@1 ($\downarrow$)	0.386 $\pm$ 0.012	0.000 $\pm$ 0.000	0.228 $\pm$ 0.018	0.226 $\pm$ 0.019	0.278 $\pm$ 0.021	0.232 $\pm$ 0.019
FDR-CE ($\downarrow$)	0.382 $\pm$ 0.013	0.000$\pm$0.000	0.225$\pm$0.018	0.225$\pm$0.017	0.270 $\pm$ 0.021	0.228$\pm$0.019
EFFICIENCY ($\uparrow$)	1.000 $\pm$ 0.000	0.000 $\pm$ 0.001	0.499 $\pm$ 0.018	0.498 $\pm$ 0.017	0.680 $\pm$ 0.020	0.513$\pm$0.019
JAVA ($\varepsilon_S = 0.3$)						
1 - PASS@1 ($\downarrow$)	0.383 $\pm$ 0.013	0.000 $\pm$ 0.000	0.240 $\pm$ 0.017	0.228 $\pm$ 0.019	0.286 $\pm$ 0.020	0.227 $\pm$ 0.020
FDR-CE ($\downarrow$)	0.379 $\pm$ 0.013	0.000$\pm$0.000	0.237$\pm$0.017	0.223$\pm$0.019	0.282 $\pm$ 0.019	0.222$\pm$0.022
EFFICIENCY ($\uparrow$)	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.502$\pm$0.019	0.483 $\pm$ 0.019	0.672 $\pm$ 0.021	0.485 $\pm$ 0.018
JAVASCRIPT ($\varepsilon_S = 0.3$)						
1 - PASS@1 ($\downarrow$)	0.374 $\pm$ 0.014	0.000 $\pm$ 0.000	0.221 $\pm$ 0.016	0.227 $\pm$ 0.025	0.278 $\pm$ 0.020	0.227 $\pm$ 0.022
FDR-CE ($\downarrow$)	0.368 $\pm$ 0.014	0.000$\pm$0.000	0.215$\pm$0.016	0.223$\pm$0.025	0.274$\pm$0.018	0.222$\pm$0.023
EFFICIENCY ($\uparrow$)	1.000 $\pm$ 0.000	0.000 $\pm$ 0.000	0.500 $\pm$ 0.018	0.518 $\pm$ 0.020	0.673$\pm$0.015	0.517 $\pm$ 0.020
PERL ($\varepsilon_S = 0.3$)						
1 - PASS@1 ($\downarrow$)	0.457 $\pm$ 0.015	0.000 $\pm$ 0.000	0.305 $\pm$ 0.017	0.205 $\pm$ 0.034	0.274 $\pm$ 0.029	0.218 $\pm$ 0.037
FDR-CE ($\downarrow$)	0.453 $\pm$ 0.015	0.000$\pm$0.000	0.302 $\pm$ 0.016	0.207$\pm$0.033	0.270 $\pm$ 0.027	0.207$\pm$0.035
EFFICIENCY ($\uparrow$)	1.000 $\pm$ 0.000	0.000 $\pm$ 0.001	0.499 $\pm$ 0.019	0.182 $\pm$ 0.031	0.397 $\pm$ 0.028	0.186$\pm$0.029

L.4. Extension Over Prior Work

Table 7. FDR-CE and efficiency for DeepSeek-R1

ε_S	FDR-CE	Efficiency
w/o SCG		
-	0.234 $\pm$ 0.012	1.000 $\pm$ 0.000
w/ SCG		
0.15	0.043 $\pm$ 0.048	0.076 $\pm$ 0.085
0.20	0.132 $\pm$ 0.015	0.515 $\pm$ 0.032
0.25	0.183 $\pm$ 0.016	0.778 $\pm$ 0.020
0.30	0.233 $\pm$ 0.011	0.995 $\pm$ 0.004

L.5. Unit Test

We add an evaluation study on *FuzzEval*, showing benefits of added unit tests and flexibility of α -code entailment. We measure the performance of generators with the conventional pass@1 metric with various α . Table L.5 demonstrates that more thorough evaluation is achievable with test cases generated by a fuzzing tool. Furthermore, pass@1 results with relaxed α -setting implies the necessity of incorporating α in *FuzzEval*.

Table 8. Pass@1 of each generator with original dataset and generated unit tests. We report pass@1 on original datasets along with added unit tests. We also measured α to show flexibility and excluded problems that encountered an error during the generation process. Furthermore, the results are measured on the subset of the original dataset, excluding 1581 problems without unit tests.

Code Generator Model	without additional unit tests	with additional unit tests			
	pass@1	pass@1	pass@1 ($\alpha = 0.1$)	pass@1 ($\alpha = 0.3$)	pass@1 ($\alpha = 0.5$)
GPT-4o	0.515	0.483	0.526	0.577	0.589
Gemini-1.5 Pro	0.526	0.480	0.523	0.566	0.578
CodeLlama-13B-Instruct	0.051	0.048	0.059	0.073	0.075

L.6. LLM-generated unit tests

Unit tests generated via other techniques could also be used in **SCG**. To support our claim, we conducted experiments utilizing unit test sets generated with the assistance of LLMs. We applied **SCG** on HumanEval+ (Liu et al., 2023) and MBPP+(Liu et al., 2023), which are publicly available datasets augmented with LLM-based methods. Table 9 demonstrates the results of our method.

Table 9. We used GPT-4o as the model. The parameters are set as $\alpha = 0.2$, $\delta_S = 0.1$, $\varepsilon_S = 0.25$, $\varepsilon_E = 0.05$. The FDR-CE satisfies the desired bound (ε_S) which demonstrates that our method is applicable on different datasets with tests generated with the assistance of LLMs.

Dataset	pass@1	FDR-CE	Efficiency
MBPP+	0.720	0.121 ± 0.112	0.423 ± 0.446
HumanEval+	0.848	0.056 ± 0.044	0.979 ± 0.029

L.7. Experiment Setup

We used 4 NVIDIA A100 80GB with 128 CPUs for code generation. We used the same environment for fuzzing and calibration.

M. Discussion

M.1. Guidelines for Parameter Selection

Our algorithm has user-specified parameters, ε_S , δ_S , α , ε_E , and $n_{\max}$. The ε_S and δ_S are the main parameters that encode the user's desired constraints on the performance of the selective generator, where the smaller values are better.

The α encodes a degree of the correctness of a generator, where $\alpha = 0$ is the ideal value. But, assuming that there is no perfect generator that exactly returns correct code, this is never achieved. We recommend to choose a small value for this when evaluating current state-of-the-art generators.

The ε_E and $n_{\max}$ are associated to the number of generated unit tests via dynamic code analysis tools, where the ideal value of ε_E is zero and $n_{\max}$ is infinity. The larger number of unit tests is definitely preferred but it sacrifices the running time of dynamic code analysis tools and evaluation.